



HCM-UTE

TRƯỜNG ĐẠI HỌC

CÔNG NGHỆ KỸ THUẬT TP. HỒ CHÍ MINH

HCMC University of Technology and Engineering

HỌC PHẦN: TƯƠNG TÁC DỮ LIỆU TRỰC QUAN

Mã học phần: IDVI333677

Giảng viên: Ths. Đoàn Minh Trí

Email liên hệ: doanminhtri@hcmute.edu.vn





HCM-UTE

HO CHI MINH CITY
UNIVERSITY OF TECHNOLOGY AND ENGINEERING

01, Vo Van Ngan street, Thu Duc ward, Ho Chi Minh City, Vietnam
hcmute.edu.vn

KHOA CÔNG NGHỆ THÔNG TIN - NGÀNH KỸ THUẬT DỮ LIỆU

Chương 3: Biểu đồ & Đồ thị Nâng cao

(Nội dung chương 7 được giảng dạy kết hợp)

Nghệ thuật Khai phá và Tương tác Dữ liệu Trực quan

Giảng viên: Ths. Đoàn Minh Trí

Email liên hệ: doanminhtri@hcmute.edu.vn

IDVI333677 | Interactive Data Visualization



NỘI DUNG CHƯƠNG 3

1. Các ô vuông và râu trên biểu đồ
2. Biểu đồ hộp
3. Histogram
4. Một số dạng biểu đồ khác
5. Kết hợp các dạng biểu đồ khác nhau
6. Bản đồ và ứng dụng của bản đồ
7. Đường tham chiếu (Reference Line) , đường thả xuống (Drop Line) và chú giải công cụ
8. Sử dụng R/Python để vẽ biểu đồ



Tại sao biểu đồ cơ bản là chưa đủ?

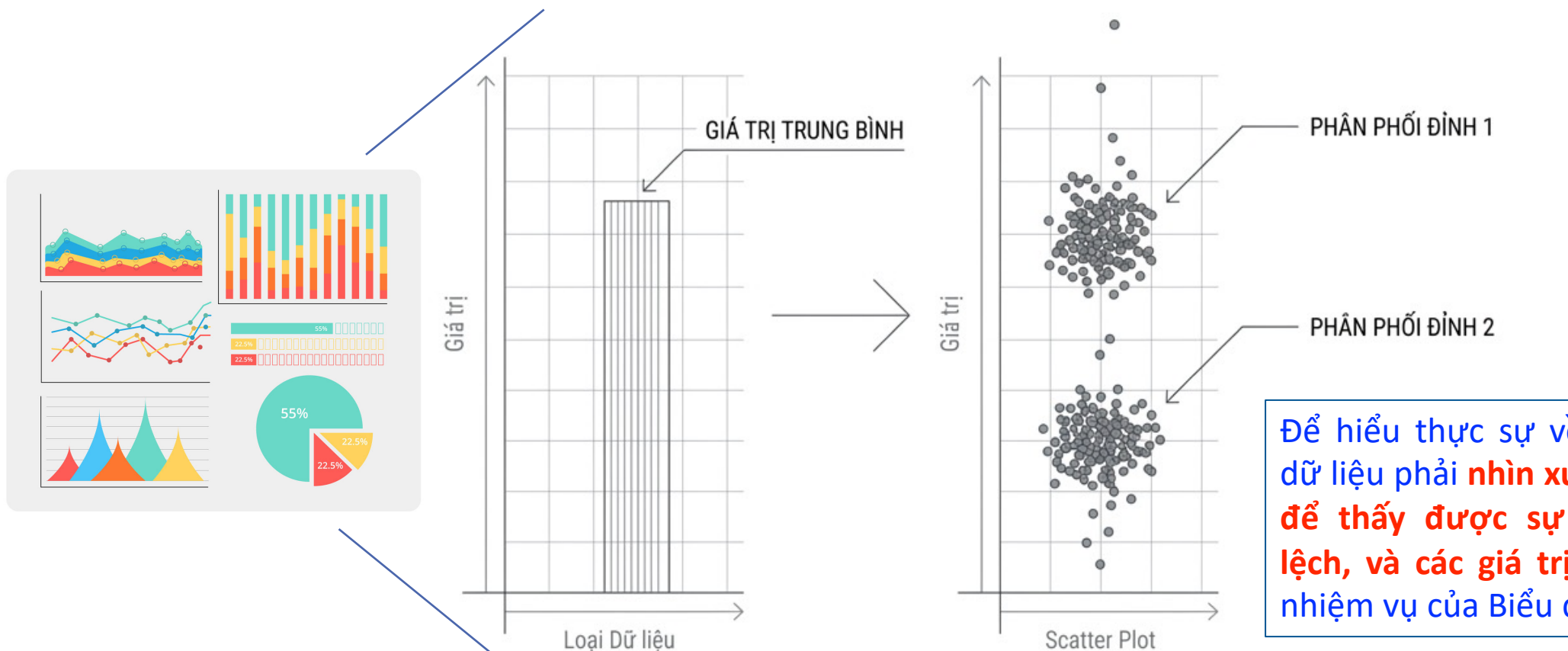




HCM-UTE

Điểm mù của giá trị

Biểu đồ cơ bản (như Bar chart) thường tóm tắt dữ liệu thành một con số duy nhất (ví dụ: Giá trị trung bình). Nhưng trong thế giới thực, dữ liệu không bao giờ đơn giản như vậy.



Khung kiến trúc của dữ liệu nâng cao

Vượt qua những con số bề mặt, chúng ta sẽ học cách nhìn sâu hơn vào cấu trúc dữ liệu.



Chiều sâu (Depth)

Box plot & Histogram: Đọc vị sự phân bố và các giá trị dị biệt.



Đa chiều (Dimensions)

Biểu đồ kết hợp & Bản đồ: Kể câu chuyện không gian và lớp lang dữ liệu.



Ngữ cảnh (Context)

Reference lines, Drop lines & Tooltips: Tăng cường tính tương tác và định hướng.



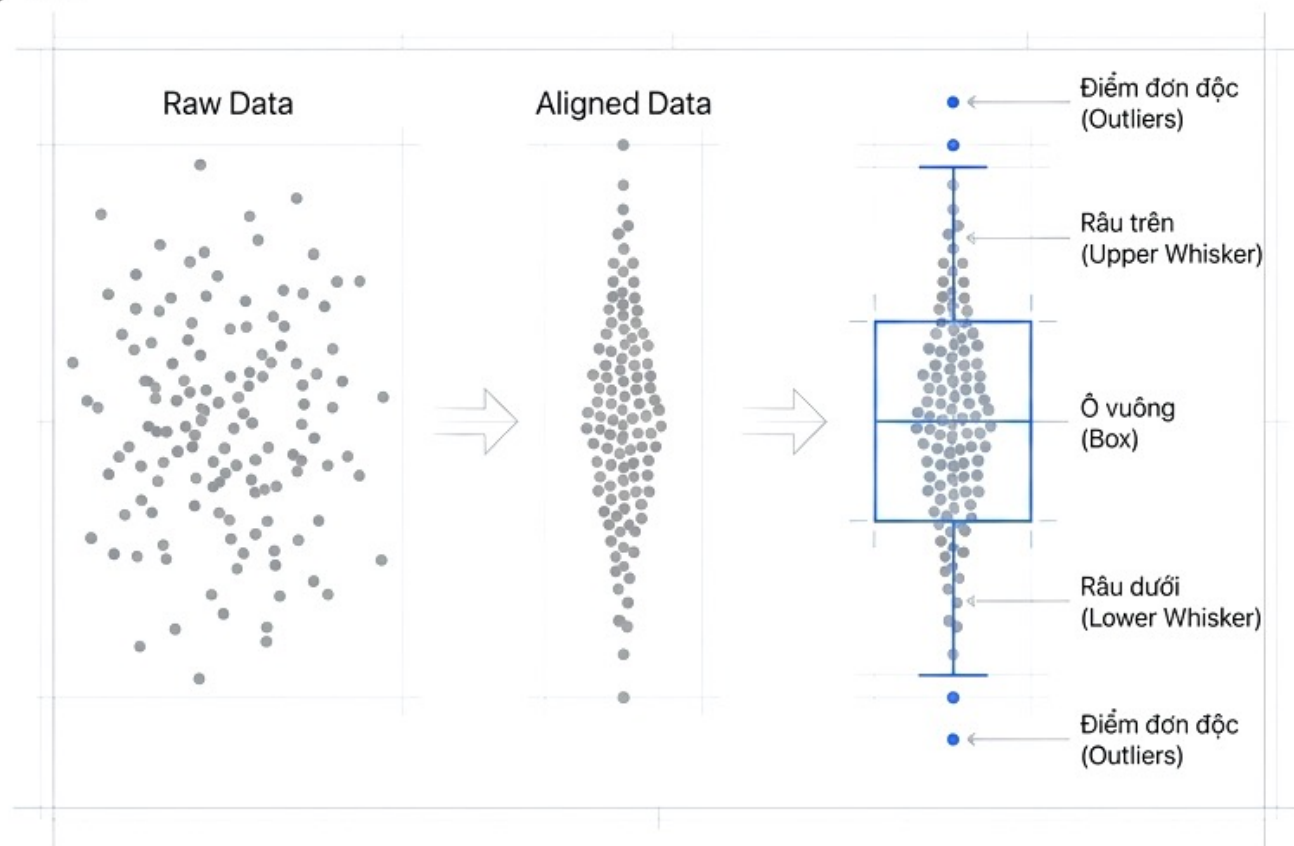
Thực thi (Execution)

R, Python & Đồ án nhóm: Biến lý thuyết thiết kế thành sản phẩm thực tế.

Bản chất của Box Plot: Tóm tắt Dữ liệu qua Hộp và Râu

Thay vì vẽ từng điểm (gây rối mắt) hoặc tính trung bình (làm mất thông tin), chúng ta nén dữ liệu thành các khối hình học mang tính thống kê:

- Ô vuông (Box): Gói gọn phần cốt lõi, nơi tập trung đa số dữ liệu.
- Râu (Whiskers): Mở rộng để thể hiện phạm vi biến thiên và ranh giới bình thường.
- Điểm đơn độc (Points): Tách biệt các giá trị bất thường.





HCM-UTE

Biểu đồ hộp (Box Plot): Đọc vị sự phân bố

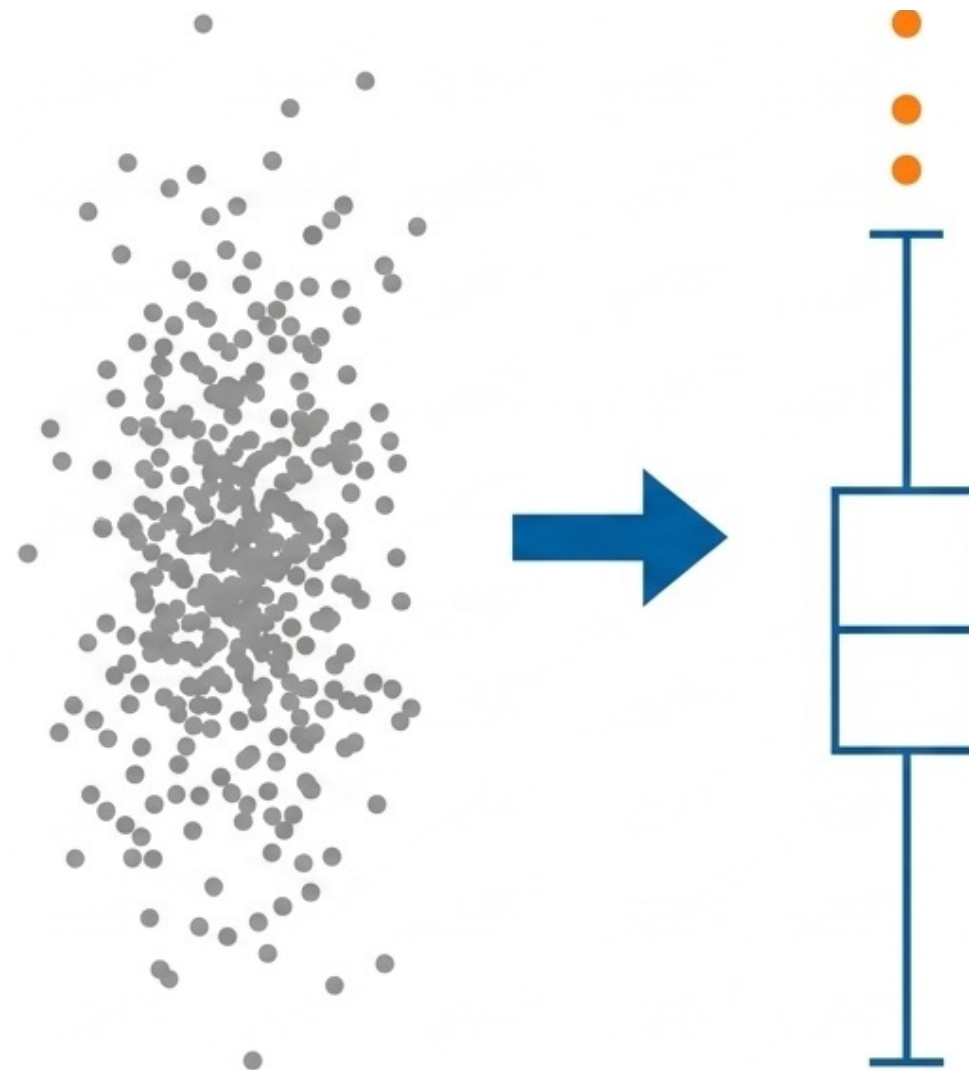
Vấn đề: Biểu đồ cột/đường thường "giấu" mất độ lệch và ngoại lệ của dữ liệu.

Giải pháp (Box Plot): Cô đọng hàng ngàn điểm dữ liệu thành 5 con số thống kê cốt lõi:

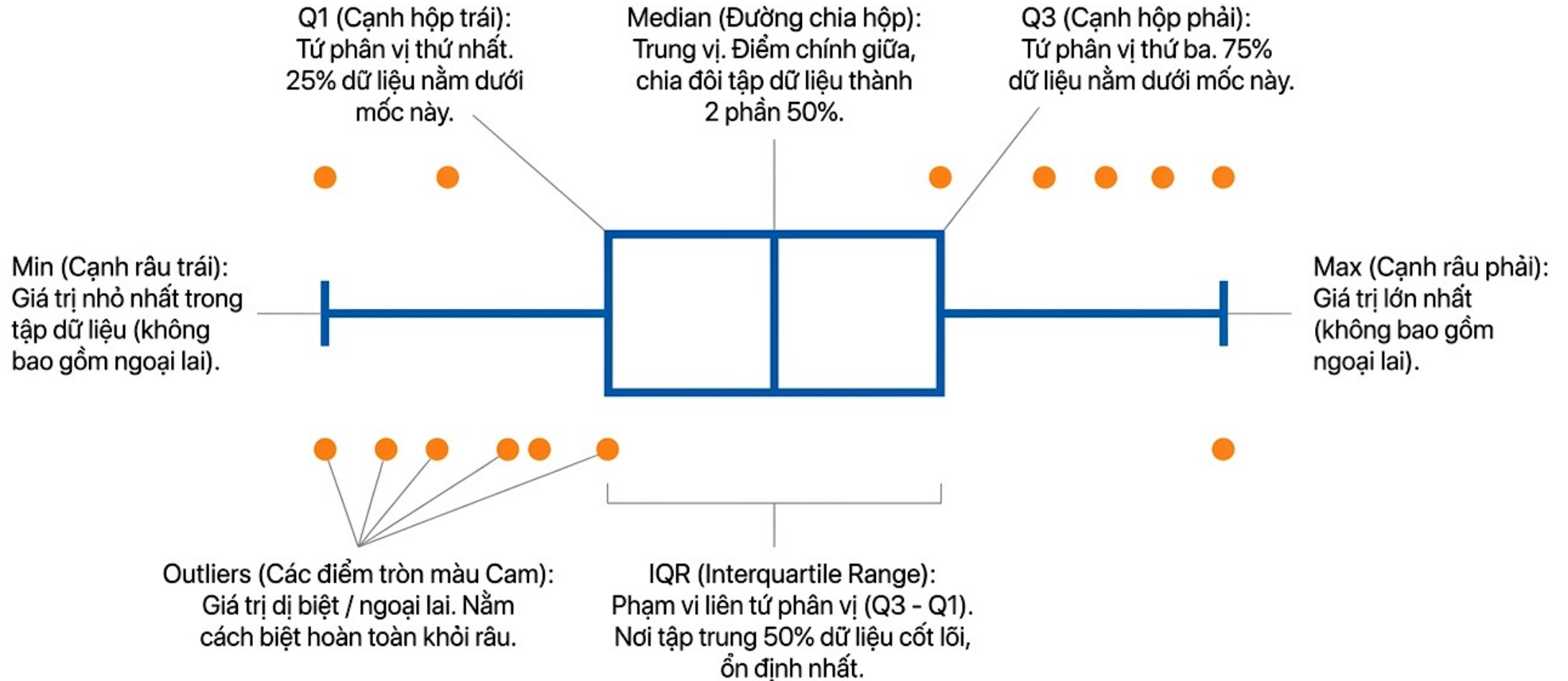
- Min & Max (*Ranh giới râu*)
- Q_1 , Q_2 (**Median**), Q_3 (*Khung hộp 50% dữ liệu*)
- Outliers (*Giá trị dị biệt*)

Sức mạnh: Nhận diện ngay lập tức **độ lệch**, **mức độ phân tán** và **điểm bất thường** chỉ trong 1 giây.

Ví dụ thực tế: So sánh phổ lương giữa các phòng ban (thay vì chỉ nhìn vào mức lương trung bình dễ bị lệch).



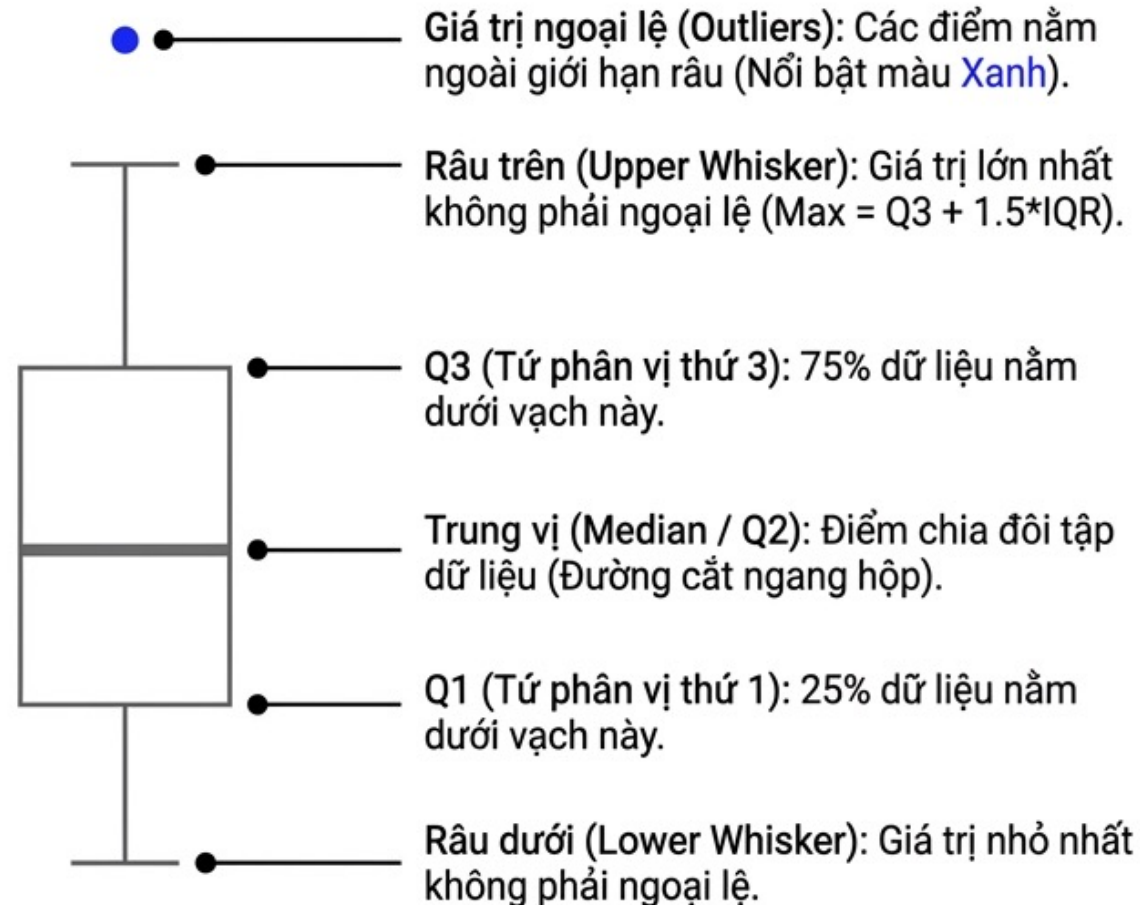
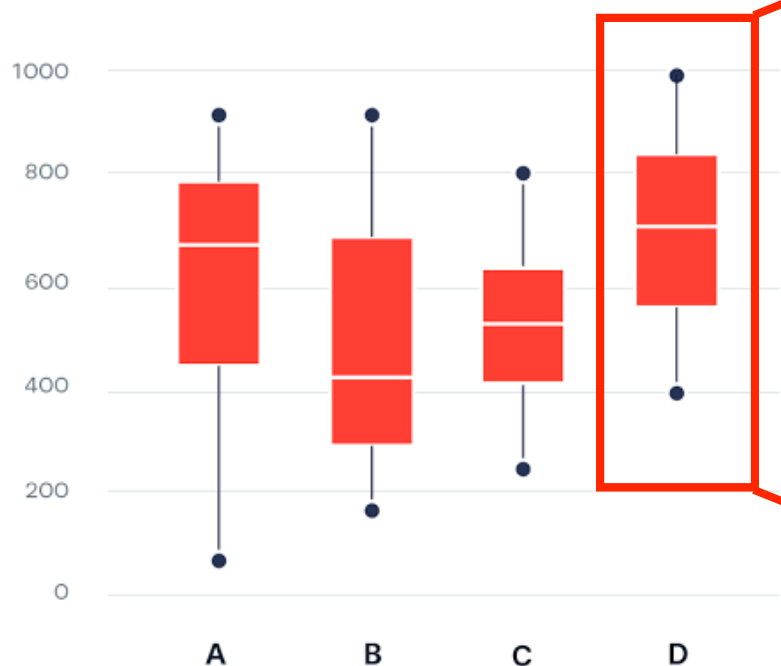
Cấu tạo biểu đồ hộp theo chiều ngang





HCM-UTE

Cấu tạo biểu đồ hộp theo chiều dọc





BÀI TẬP: PHÂN TÍCH TỨ PHÂN VỊ VÀ VẼ BIỂU ĐỒ BOX PLOT

Đề bài:

Kết quả kiểm tra môn Thống kê của một nhóm gồm **11 sinh viên** được ghi nhận như sau:

9	4	7	5	8	3	7	10	6	8	7
---	---	---	---	---	---	---	----	---	---	---

Yêu cầu:

1. Tìm các mốc tứ phân vị (Q_1 , Q_2 , Q_3) của tập dữ liệu trên (nêu rõ các bước tính toán).
2. Xác định tập hợp 5 số đặc trưng (Five-summary) để chuẩn bị vẽ biểu đồ.
3. Tính Khoảng tứ phân vị (IQR) của dãy điểm số.
4. Kiểm tra xem tập dữ liệu có tồn tại giá trị outlier nào không.
5. Vẽ biểu đồ Box Plot bằng tay lên bảng (Số ghi chú) dựa trên các thông số đã tìm được.

BÀI TẬP: PHÂN TÍCH TỨ PHÂN VỊ VÀ VẼ BIỂU ĐỒ BOX PLOT

Câu 1

Bước 1: Sắp xếp dãy số theo thứ tự tăng dần

Dãy điểm số gốc: 9, 4, 7, 5, 8, 3, 7, 10, 6, 8, 7

Sau khi sắp xếp: **3, 4, 5, 6, 7, 7, 7, 8, 8, 9, 10**

Bước 2: Tìm Trung vị (Q2) để chia đôi dãy số

Vì dãy có 11 số (số lẻ), số ở chính giữa là số ở vị trí thứ 6.

- Dãy số: 3, 4, 5, 6, 7, **[7]**, 7, 8, 8, 9, 10

- **Trung vị (Q2) = 7** (Chia dữ liệu thành 2 nửa, mỗi nửa có 5 số).

Bước 3: Tìm Q1 và Q3 bằng cách chia đôi tiếp hai nửa

- **Tìm Q1:** Lấy trung vị của nửa số bên trái (3, 4, 5, 6, 7). Số ở giữa là 5. → **Q1 = 5**

- **Tìm Q3:** Lấy trung vị của nửa số bên phải (7, 8, 8, 9, 10). Số ở giữa là 8. → **Q3 = 8**

BÀI TẬP: PHÂN TÍCH TỨ PHÂN VỊ VÀ VẼ BIỂU ĐỒ BOX PLOT

Câu 2: Xác định tập hợp 5 số đặc trưng

- Giá trị nhỏ nhất (Min) = 3
- Tứ phân vị thứ nhất (Q1) = 5
- Trung vị (Q2) = 7
- Tứ phân vị thứ ba (Q3) = 8
- Giá trị lớn nhất (Max) = 10



BÀI TẬP: PHÂN TÍCH TỨ PHÂN VỊ VÀ VẼ BIỂU ĐỒ BOX PLOT

Câu 3: Tính Khoảng tứ phân vị (IQR)

Dựa vào kết quả đã tìm ở câu trước ($Q1 = 5$) và ($Q3 = 8$):

$$IQR = Q3 - Q1 = 8 - 5 = 3$$

Câu 4: Kiểm tra giá trị Outlier

Bước 1: Tính các mốc chặn

$$\text{Chặn dưới: } LF = Q1 - 1.5 * IQR = 5 - 1.5 * 3 = 5 - 4.5 = 0.5$$

$$\text{Chặn trên: } UF = Q3 + 1.5 * IQR = 8 + 1.5 * 3 = 8 + 4.5 = 12.5$$

→ Vùng dữ liệu bình thường hợp lệ phải nằm trong khoảng từ **0.5 đến 12.5**

Bước 2: So sánh tập dữ liệu với hàng rào

Nhìn lại dãy số đã sắp xếp: 3, 4, 5, 6, 7, 7, 7, 8, 8, 9, 10

Giá trị nhỏ nhất là 3 (vẫn lớn hơn 0.5 → không bị ngoại lệ dưới).

Giá trị lớn nhất là 10 (vẫn nhỏ hơn 12.5 → không bị ngoại lệ trên).

Kết luận: Tập dữ liệu này không có giá trị outlier



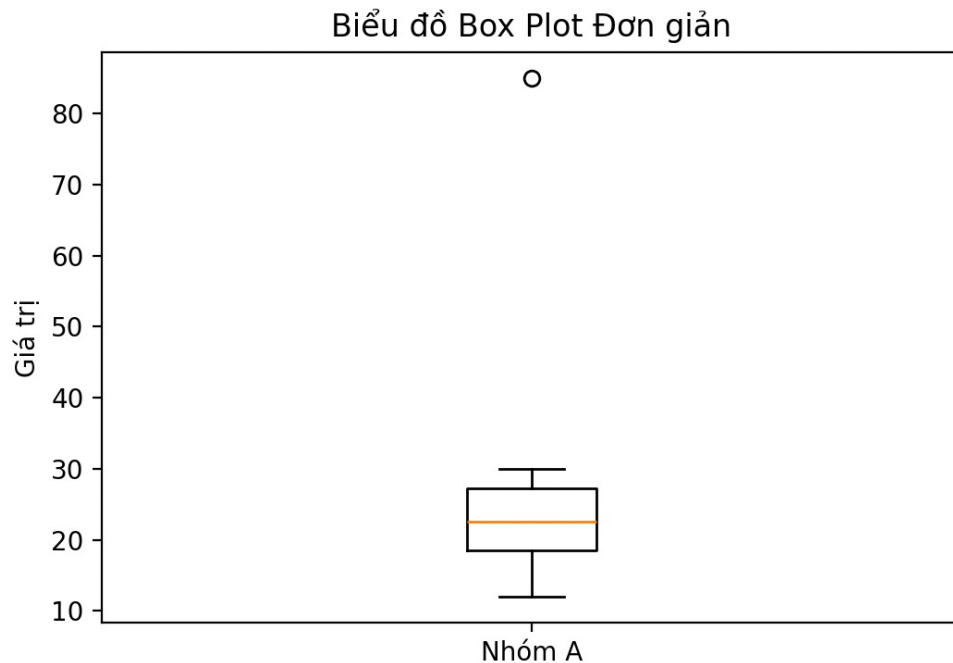
BÀI TẬP: PHÂN TÍCH TƯ PHÂN VỊ VÀ VẼ BIỂU ĐỒ BOX PLOT

Câu 5: Vẽ biểu đồ Box Plot bằng tay

- 1.Trục tọa độ:** Phải vẽ một trục số nằm ngang (hoặc thẳng đứng) có chia vạch đều nhau từ 2 đến 11 để định vị chính xác các điểm.
- 2.Thân hộp:** Hộp phải bắt đầu từ vạch số **5 (Q1)** và kết thúc ở vạch số **8 (Q3)**.
- 3.Đường trung vị:** Đoạn thẳng đứng cắt ngang bên trong hộp ngay tại vạch số **7 (Q2)**.
- 4.Râu biểu đồ:** Hai đoạn thẳng (râu) kéo dài từ cạnh hộp ra hai bên: râu trái chạm vạch số **3 (Min)**, râu phải chạm vạch số **10 (Max)**.

Box Plot: Trên Matplotlib*

Box Plot 1 nhóm dữ liệu



```
import matplotlib.pyplot as plt
```

```
# 1. Khởi tạo dữ liệu mẫu
```

```
data = [12, 15, 18, 20, 22, 23, 25, 28, 30, 85]
```

```
# 85 là outlier
```

```
# 2. Vẽ biểu đồ Box Plot
```

```
plt.figure(figsize=(6, 4))
```

```
plt.boxplot(data)
```

```
# 3. Thêm tiêu đề và nhãn
```

```
plt.title('Biểu đồ Box Plot Đơn giản')
```

```
plt.ylabel('Giá trị')
```

```
plt.xticks([1], ['Nhóm A'])
```

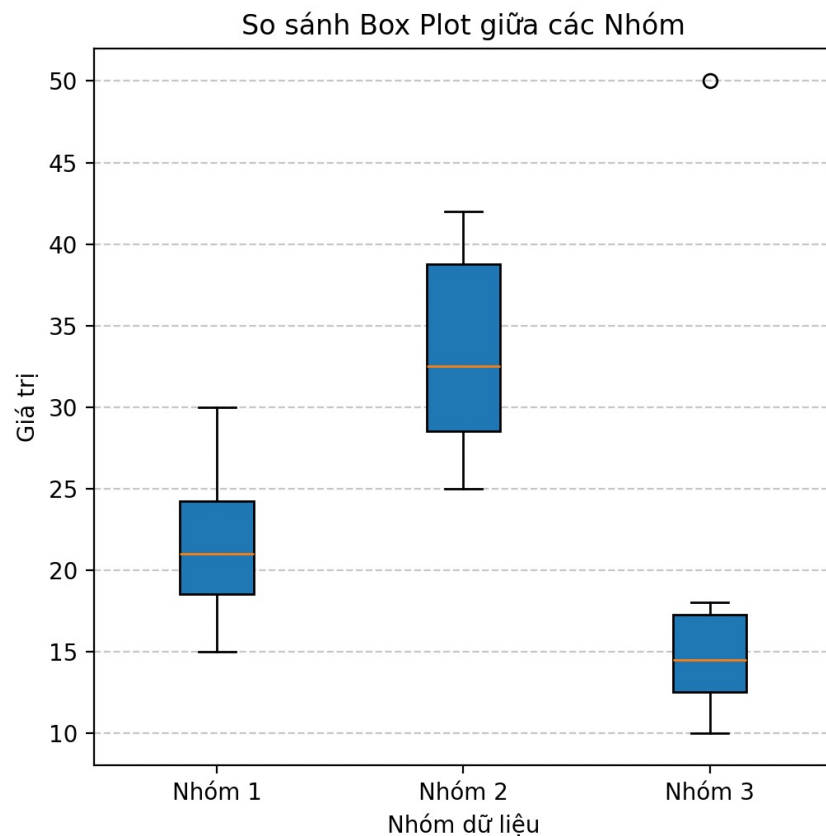
```
# 4. Hiển thị biểu đồ
```

```
plt.show()
```

**pip install matplotlib*

Box Plot: Trên Matplotlib*

Box Plot so sánh nhiều nhóm dữ liệu



```
import matplotlib.pyplot as plt
```

```
# Dữ liệu cho 3 nhóm khác nhau
```

```
group1 = [15, 18, 20, 22, 25, 30]
```

```
group2 = [25, 28, 30, 35, 40, 42]
```

```
group3 = [10, 12, 14, 15, 18, 50]
```

```
data_all = [group1, group2, group3]
```

```
plt.figure(figsize=(7, 4))
```

```
# patch_artist=True giúp tô màu cho hộp.
```

```
#vert=False nếu muốn xoay ngang
```

```
plt.boxplot(data_all, patch_artist=True,  
            tick_labels=['Nhóm 1', 'Nhóm 2', 'Nhóm 3'])
```

```
plt.title('So sánh Box Plot giữa các Nhóm')
```

```
plt.xlabel('Nhóm dữ liệu')
```

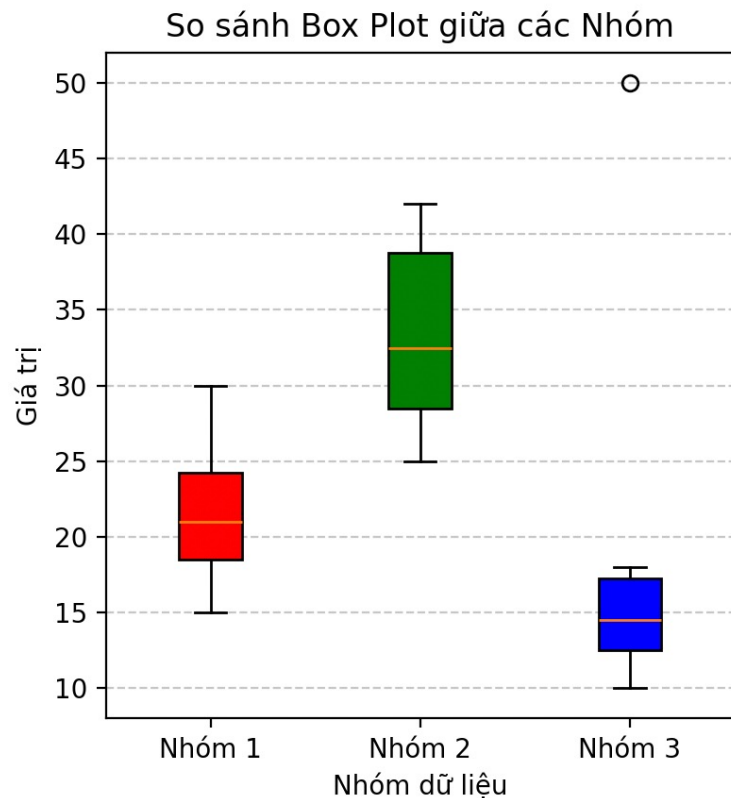
```
plt.ylabel('Giá trị')
```

```
plt.grid(axis='y', linestyle='--', alpha=0.7)
```

```
plt.show()
```

Box Plot: Trên Matplotlib*

Box Plot so sánh nhiều nhóm dữ liệu



```
import matplotlib.pyplot as plt
```

```
# Dữ liệu cho 3 nhóm khác nhau
```

```
group1 = [15, 18, 20, 22, 25, 30]
```

```
group2 = [25, 28, 30, 35, 40, 42]
```

```
group3 = [10, 12, 14, 15, 18, 50]
```

```
data_all = [group1, group2, group3]
```

```
cl=["red", "green", "blue"]
```

```
plt.figure(figsize=(7, 4))
```

```
#patch_artist=True giúp tô màu cho hộp.
```

```
#vert=False nếu muốn xoay ngang
```

```
bdo=plt.boxplot(data_all, patch_artist=True,  
                tick_labels=['Nhóm 1', 'Nhóm 2', 'Nhóm 3'])
```

```
#set_facecolor cá nhân hóa màu theo từng hộp
```

```
for patch, cl in zip(bdo["boxes"],cl):  
    patch.set_facecolor(cl)
```

```
plt.title('So sánh Box Plot giữa các Nhóm')
```

```
plt.xlabel('Nhóm dữ liệu')
```

```
plt.ylabel('Giá trị')
```

```
plt.grid(axis='y', linestyle='--', alpha=0.7)
```

```
plt.show()
```

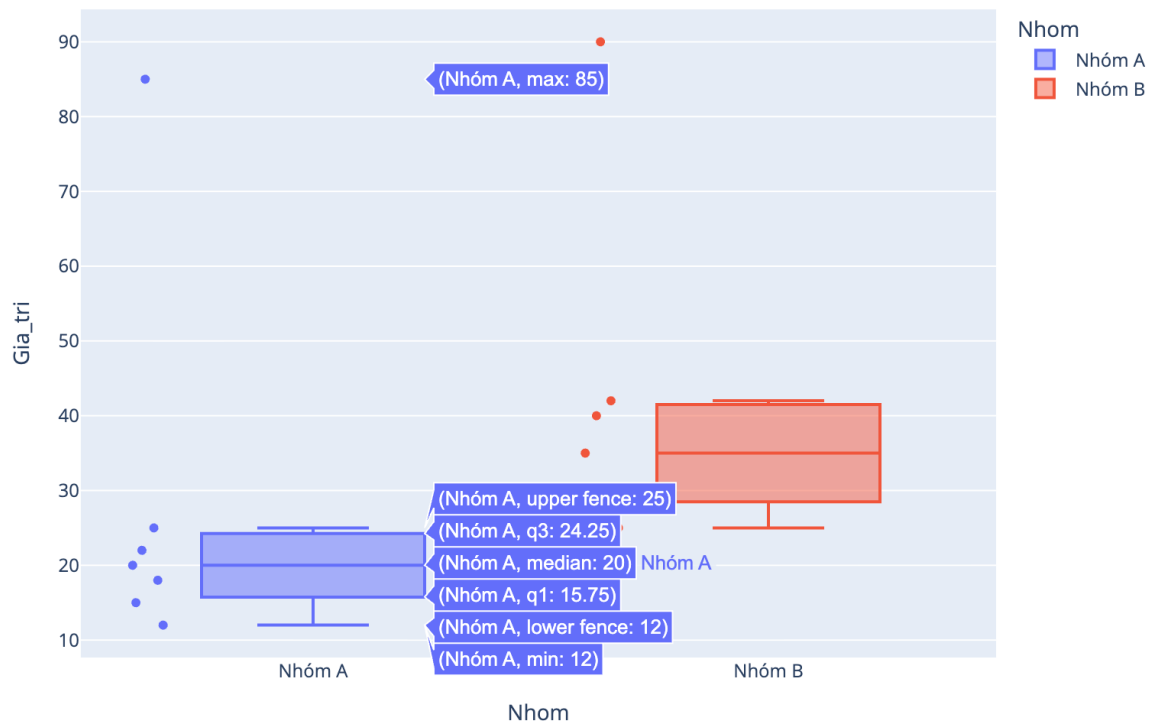
Box Plot: Trên Matplotlib*

Một số tùy chỉnh nhanh:

Tham số	Giá trị	Tác dụng
vert	False	<i>Xoay ngang biểu đồ (mặc định là True - dựng đứng).</i>
patch_artist	True	<i>Cho phép tô màu nền thân hộp.</i>
notch	True	<i>Tạo kiểu thắt eo tại vị trí Trung vị (Median).</i>
showmeans	True	<i>Hiển thị thêm mốc giá trị Trung bình (Mean).</i>
showfliers	False	<i>Ẩn toàn bộ các điểm ngoại lệ (Outliers).</i>
widths	0.3 - 0.8	<i>Điều chỉnh độ rộng của thân hộp.</i>

Box Plot tương tác: Trên Plotly*

Biểu đồ Box Plot Tương tác (Plotly)



```
import pandas as pd
import plotly.express as px
```

1. Khởi tạo dữ liệu mẫu

```
df = pd.DataFrame({
    'Nhom': ['Nhóm A']*7 + ['Nhóm B']*7,
    'Gia_tri': [12, 15, 18, 20, 22, 25, 85,
                # Nhóm A (có 1 outlier là 85)
                25, 28, 30, 35, 40, 42, 90]
    # Nhóm B (có 1 outlier là 90)
})
```

2. Vẽ Box Plot tương tác

```
fig = px.box(
    df,
    x='Nhom',
    y='Gia_tri',
    color='Nhom',
    points="all",
    hover_data=['Gia_tri'],
    title="Biểu đồ Box Plot Tương tác (Plotly)"
)
```

```
fig.show()
```

**pip install plotly*

Box Plot tương tác: Trên Plotly

Một số tùy chỉnh nhanh:

Tham số	Giá trị gợi ý	Tác dụng tương tác
points	'all', 'outliers', False	Hiển thị toàn bộ các điểm dữ liệu thực tế bên cạnh hộp để soi từng giá trị.
boxmean	True hoặc 'sd'	Hiển thị thêm mốc Giá trị Trung bình (Mean) và Độ lệch chuẩn (Standard Deviation).
orientation	'h'	Chuyển biểu đồ thành dạng nằm ngang.
notched	True	Tạo điểm thắt eo (notch) tại vị trí Trung vị.

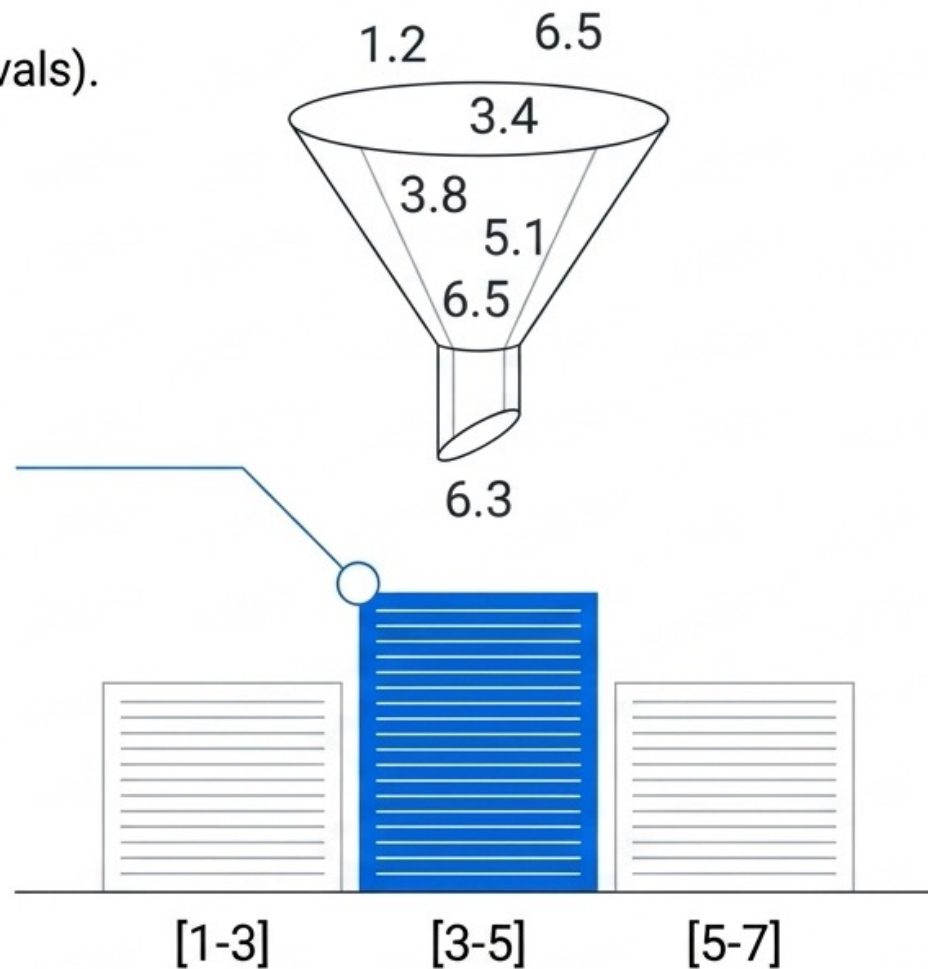
Biểu đồ phân phối tần suất (Histogram)

Biến liên tục thành các khoảng gián đoạn (Bins/Intervals).

- Khái niệm: Biểu diễn tần suất xuất hiện của các giá trị trong một tập dữ liệu liên tục.

- Cấu trúc (Bins): Trục X được chia thành các khoảng (intervals) có độ rộng bằng nhau. Độ cao của cột thể hiện số lượng điểm dữ liệu rơi vào khoảng đó.

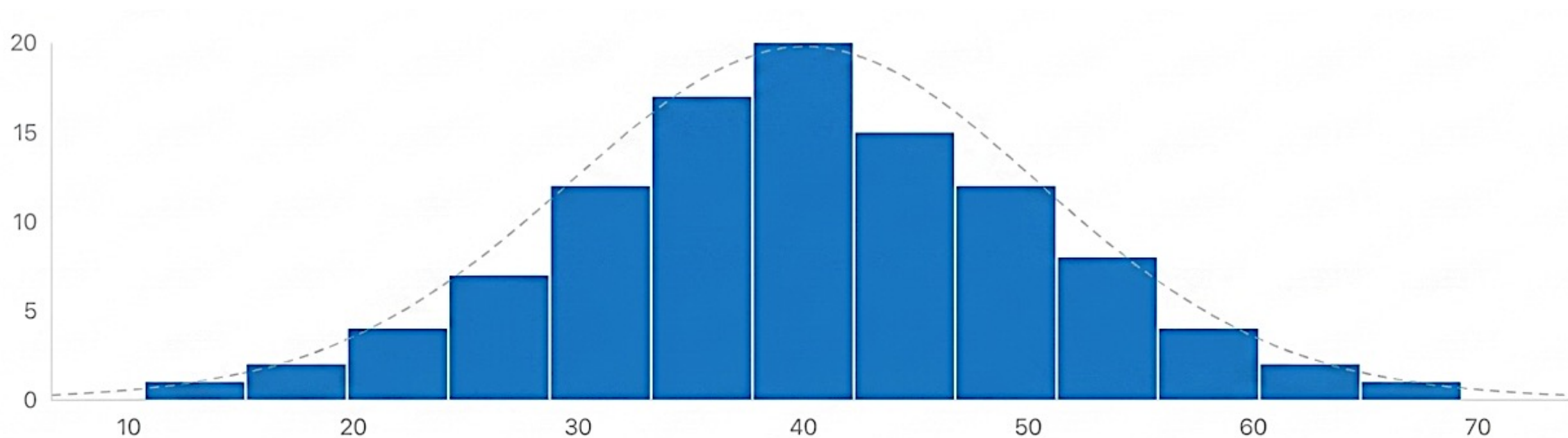
- Ý nghĩa: Trực quan hóa hình dáng của phân phối (Chuẩn, lệch trái, lệch phải, hoặc đa đỉnh).





HCM-UTE

Biểu đồ phân phối tần suất (Histogram)



Hiển thị hình dáng và độ hội tụ thực sự của dữ liệu

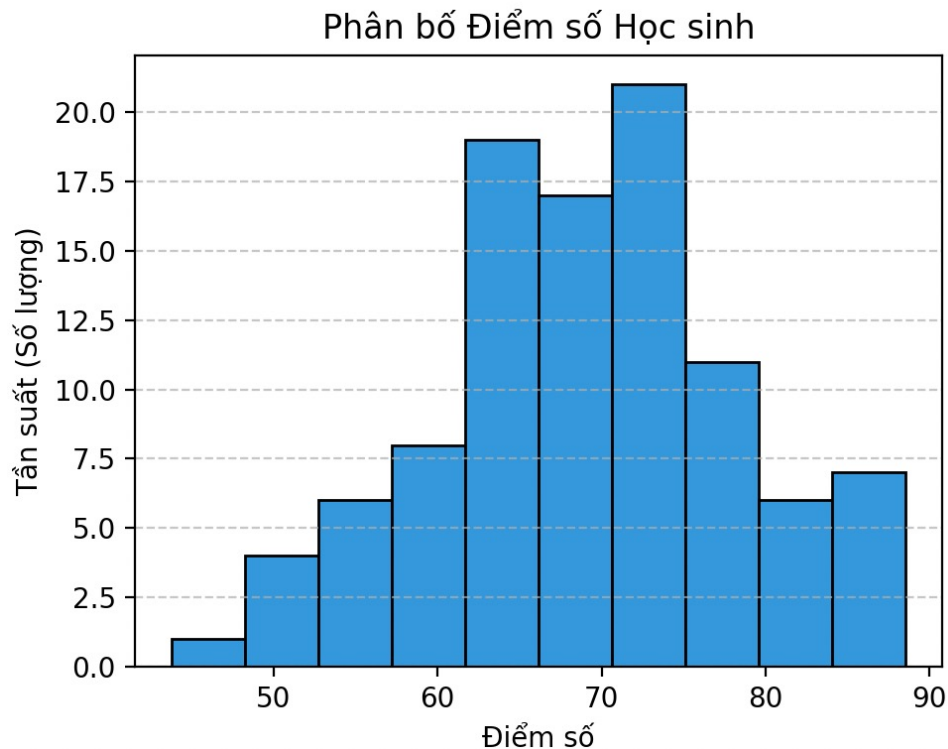
CHÚ Ý

Đừng nhầm lẫn: Histogram vs. Bar Chart

	 Biểu đồ cột (Bar Chart)	 Biểu đồ tần suất (Histogram)
Bản chất dữ liệu	Phân loại / Rời rạc (VD: Quốc gia, Sản phẩm, Phòng ban)	Định lượng / Liên tục (VD: Tuổi, Chiều cao, Thời gian)
Khoảng cách cột	CÓ khoảng trống giữa các cột (thể hiện sự biệt lập)	KHÔNG có khoảng trống (dữ liệu là một dải liên tục)
Tính linh hoạt	Có thể tự do sắp xếp lại thứ tự cột (cao xuống thấp)	Không thể đổi thứ tự (Phải tuân thủ nghiêm ngặt trục số)
Mục đích cốt lõi	So sánh giá trị tuyệt đối giữa các nhóm độc lập	Xem xét hình dáng, mật độ và độ hội tụ của một tập dữ liệu

Histogram: Trên Matplotlib*

Vẽ Histogram 1 nhóm dữ liệu



```
import matplotlib.pyplot as plt
import numpy as np
```

```
# 1. Khởi tạo dữ liệu mẫu (ví dụ: điểm số của 100 học sinh)
np.random.seed(42)
data = np.random.normal(loc=70, scale=10, size=100)
```

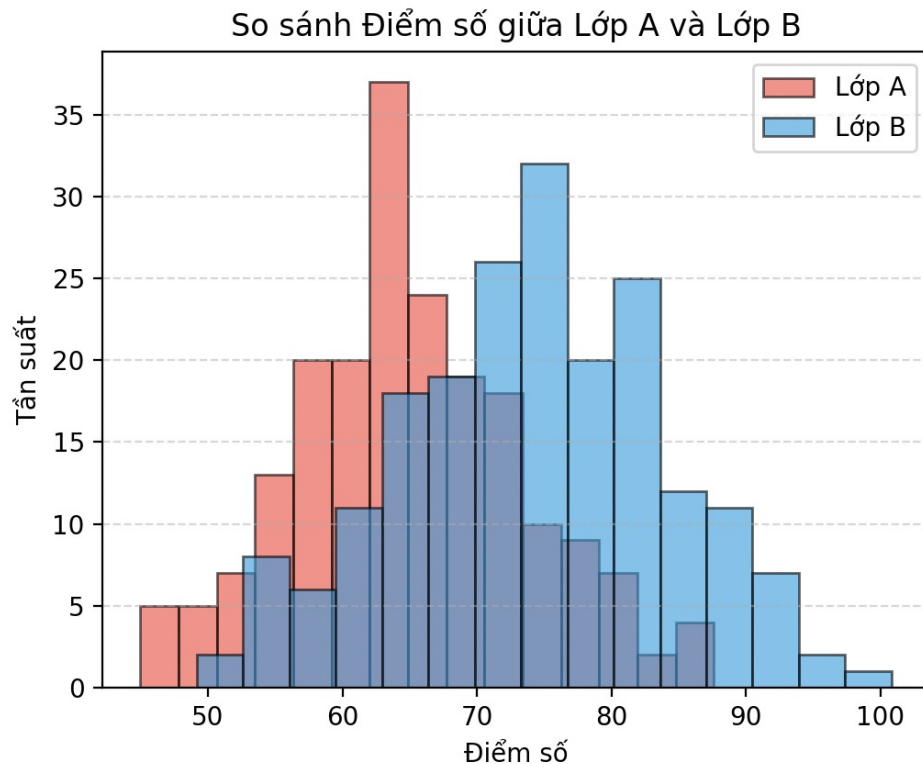
```
# 2. Vẽ biểu đồ Histogram
plt.figure(figsize=(7, 4))
plt.hist(data, bins=10, color='#3498db', edgecolor='black')
```

```
# 3. Thêm tiêu đề và nhãn trục
plt.title('Phân bố Điểm số Học sinh')
plt.xlabel('Điểm số')
plt.ylabel('Tần suất (Số lượng)')
plt.grid(axis='y', linestyle='--', alpha=0.7)
```

```
# 4. Hiển thị biểu đồ
plt.show()
```

Histogram : Trên Matplotlib*

Vẽ so sánh 2 phân bố dữ liệu
trên cùng 1 biểu đồ



```
import matplotlib.pyplot as plt
import numpy as np
```

```
group_A = np.random.normal(loc=65, scale=8, size=200)
group_B = np.random.normal(loc=75, scale=10, size=200)
```

```
plt.figure(figsize=(8, 4.5))
```

```
# Sử dụng alpha để làm mờ
```

```
#giúp thấy rõ phần giao nhau giữa 2 nhóm
```

```
plt.hist(group_A, bins=15, alpha=0.6, label='Lớp A',
         color='#e74c3c', edgecolor='black')
```

```
plt.hist(group_B, bins=15, alpha=0.6, label='Lớp B',
         color='#3498db', edgecolor='black')
```

```
plt.title('So sánh Điểm số giữa Lớp A và Lớp B')
```

```
plt.xlabel('Điểm số')
```

```
plt.ylabel('Tần suất')
```

```
plt.legend(loc='upper right')
```

```
plt.grid(axis='y', linestyle='--', alpha=0.5)
```

```
plt.show()
```



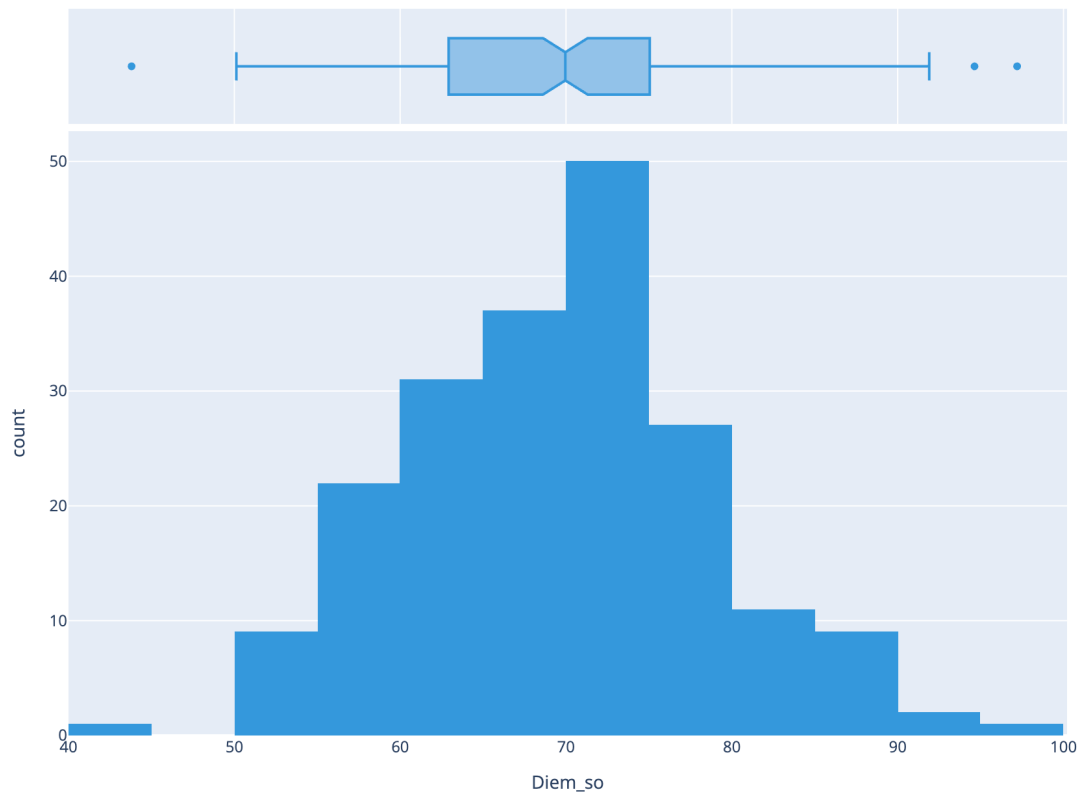
Histogram: Trên Matplotlib*

Một số tùy chỉnh nhanh:

Tham số	Giá trị gợi ý	Tác dụng
bins	10, 20 hoặc danh sách [50, 60, 70, 80, 90, 100]	Số lượng cột (thùng chứa dữ liệu) hoặc thiết lập ranh giới khoảng chia cụ thể.
color	'#2ecc71', 'orange'	Màu nền của các cột.
edgecolor	'black', 'white'	Màu đường viền bao quanh cột (giúp các cột phân tách rõ ràng).
density	True	Chuyển trục Y từ số đếm tần suất sang Mật độ xác suất (tổng diện tích các cột = 1).
alpha	0.5 đến 1.0	Độ mờ của cột (rất hữu ích khi vẽ nhiều Histogram đè lên nhau).
cumulative	True	Vẽ biểu đồ tần suất tích lũy (cộng dồn các khoảng trước đó).

Histogram tương tác: Trên Plotly*

Phân bố Điểm số Tương tác



```
import numpy as np
import pandas as pd
import plotly.express as px
```

1. Khởi tạo dữ liệu mẫu

```
np.random.seed(42)
df = pd.DataFrame({
    'Diem_so': np.random.normal(loc=70, scale=10, size=200)
})
```

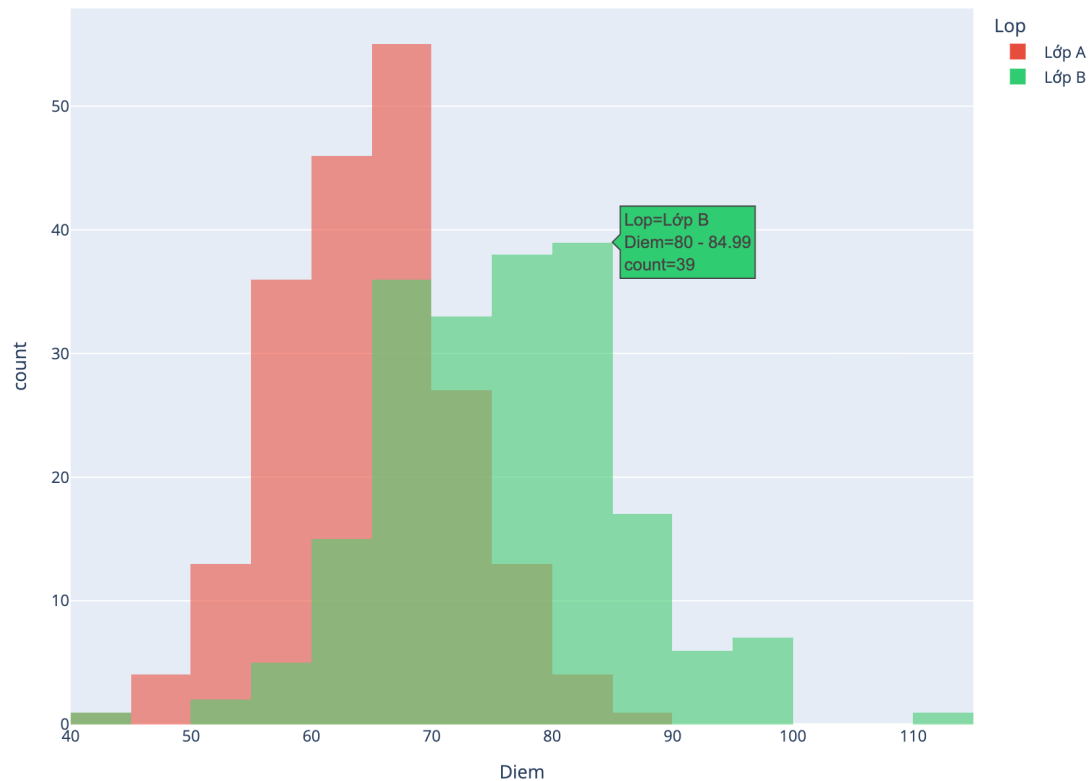
2. Vẽ Histogram tương tác

```
fig = px.histogram(
    df,
    x='Diem_so',
    nbins=15,
    title="Phân bố Điểm số Tương tác",
    # Thêm biểu đồ Box Plot phụ ở ngay bên trên
    marginal="box",
    color_discrete_sequence=['#3498db']
)
```

```
fig.show()
```

Histogram tương tác: Trên Plotly*

So sánh Phân bố Điểm số giữa Lớp A và Lớp B



```
import numpy as np
import pandas as pd
import plotly.express as px
```

Tạo dữ liệu 2 lớp

```
np.random.seed(42)
df_a = pd.DataFrame({'Diem': np.random.normal(65, 8, 200), 'Lop': 'Lớp A'})
df_b = pd.DataFrame({'Diem': np.random.normal(75, 10, 200), 'Lop': 'Lớp B'})
df_all = pd.concat([df_a, df_b])
```

Vẽ Histogram so sánh

```
fig = px.histogram(
    df_all,
    x='Diem',
    color='Lop',
    barmode='overlay',
    opacity=0.6,
    nbins=20,
    color_discrete_map={'Lớp A': '#e74c3c', 'Lớp B': '#2ecc71'},
    title="So sánh Phân bố Điểm số giữa Lớp A và Lớp B"
)

fig.show()
```

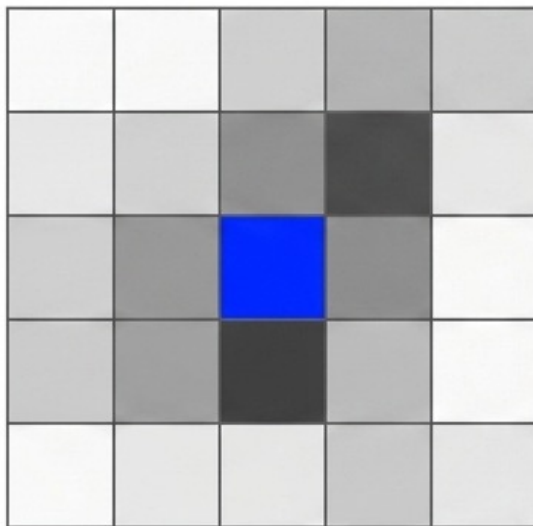
Đè 2 màu lên nhau để dễ so sánh
Chỉnh độ mờ để nhìn xuyên qua

Histogram tương tác: Trên Plotly

Một số tùy chỉnh nhanh:

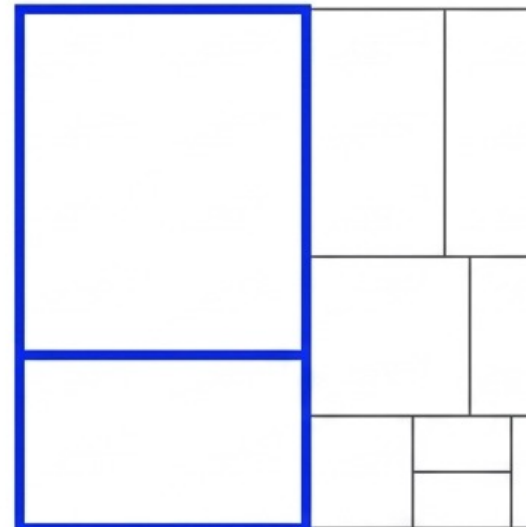
Tham số	Giá trị gợi ý	Tác dụng
nbins	10, 20, 30	Điều chỉnh số lượng khoảng chia (khoảng chứa).
marginal	'box', 'violin', 'rug'	Vẽ thêm biểu đồ phụ phía trên lề để soi kỹ phân bố và outlier.
histnorm	'percent', 'probability', 'density'	Chuyển trục Y từ số lượng đếm sang tỷ lệ phần trăm hoặc mật độ.
barmode	'overlay', 'group', 'stack'	Quy định cách hiển thị khi so sánh nhiều nhóm dữ liệu.

Một số loại biểu đồ khác thường dùng trong IDV



Heatmap - Biểu đồ Nhiệt

- **Cơ chế:** Sử dụng cường độ màu sắc trên một ma trận 2D để mã hóa giá trị dữ liệu.
- **Ứng dụng:** Tuyệt vời để phát hiện các cụm (clusters), phân tích tương quan (correlation matrix), hoặc mật độ theo thời gian.



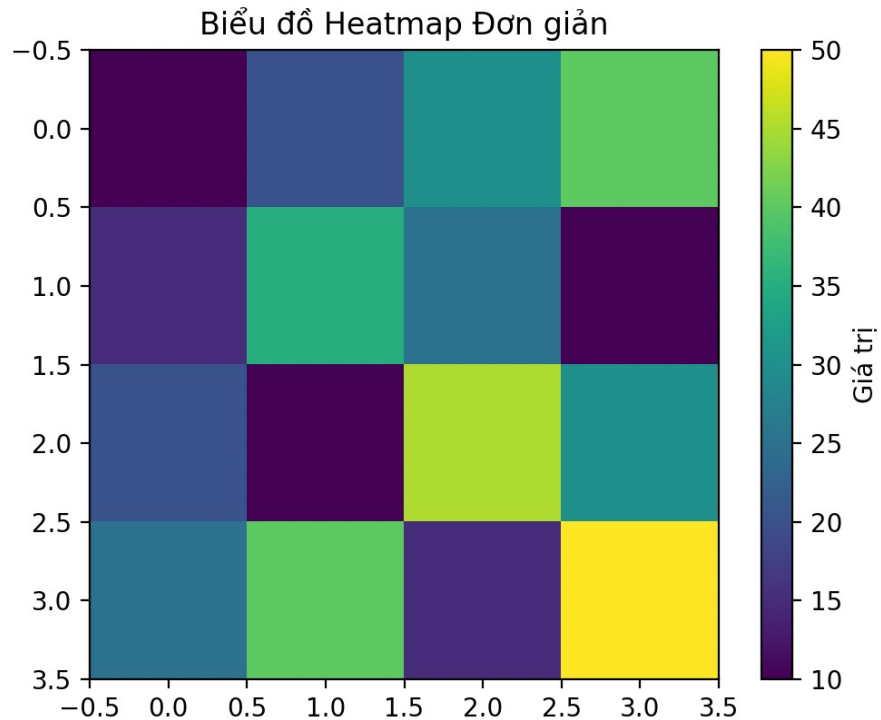
Treemap - Biểu đồ Cây

- **Cơ chế:** Sử dụng diện tích của các hình chữ nhật lồng nhau để thể hiện tỷ lệ.
- **Ứng dụng:** Hiển thị cấu trúc dữ liệu phân cấp (hierarchical) và tỷ trọng của từng phần so với tổng thể mà không tốn diện tích như biểu đồ tròn (Pie chart).

Mục đích thường gặp: Trực quan hóa mật độ và phân cấp dữ liệu

Heatmap: Trên Matplotlib*

Vẽ Heatmap đơn giản nhất



```
import matplotlib.pyplot as plt
import numpy as np
```

```
# 1. Khởi tạo mảng dữ liệu 2 chiều (ví dụ ma trận 4x4)
```

```
data = np.array([
    [10, 20, 30, 40],
    [15, 35, 25, 10],
    [20, 10, 45, 30],
    [25, 40, 15, 50]
])
```

```
# 2. Vẽ biểu đồ Heatmap
```

```
plt.figure(figsize=(6, 4.5))
```

```
heatmap = plt.imshow(data, cmap='viridis') # cmap định nghĩa bảng màu
```

```
# 3. Thêm thanh đo giá trị màu (Colorbar) và tiêu đề
```

```
plt.colorbar(heatmap, label='Giá trị')
```

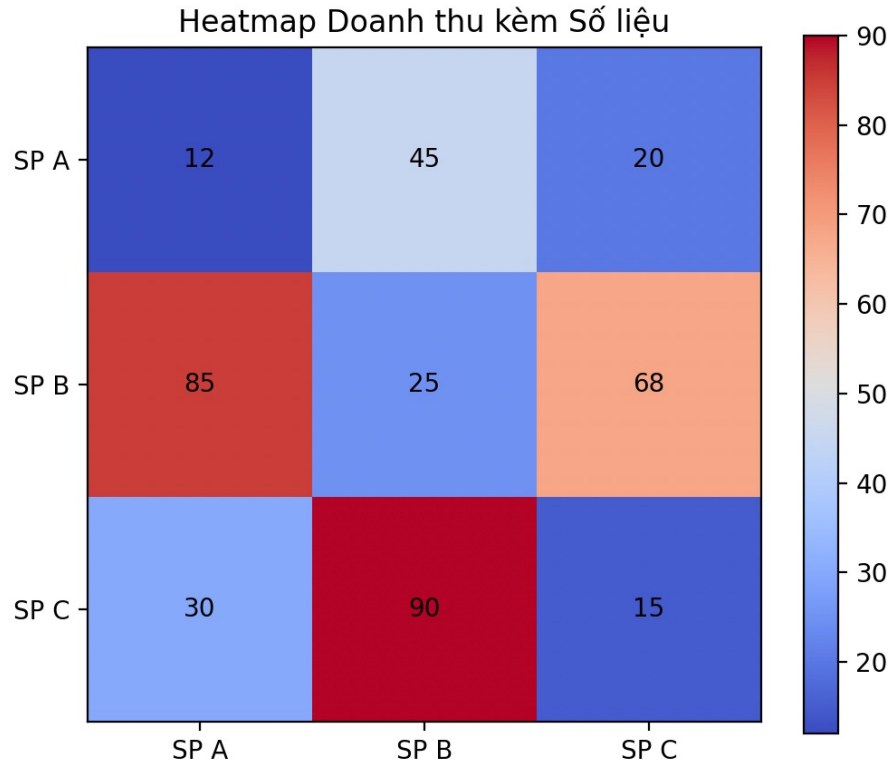
```
plt.title('Biểu đồ Heatmap Đơn giản')
```

```
# 4. Hiển thị biểu đồ
```

```
plt.show()
```

Heatmap : Trên Matplotlib*

Thêm nhãn hàng/cột và ghi số
liệu trực tiếp lên từng ô



```
import matplotlib.pyplot as plt
import numpy as np
```

```
# Dữ liệu ma trận tương quan/doanh thu 3x3
data = np.array([[12, 45, 20],
                 [85, 25, 68],
                 [30, 90, 15]])
```

```
categories = ['SP A', 'SP B', 'SP C']
```

```
plt.figure(figsize=(6, 5))
plt.imshow(data, cmap='coolwarm')
plt.colorbar()
```

```
# Gán nhãn tên sản phẩm vào 2 trục
plt.xticks(range(3), categories)
plt.yticks(range(3), categories)
```

```
# Vòng lặp chèn số trực tiếp vào trung tâm từng ô
for i in range(data.shape[0]):
    for j in range(data.shape[1]):
        plt.text(j, i, str(data[i, j]),
                 ha='center', va='center', color='black')
```

```
plt.title('Heatmap Doanh thu kèm Số liệu')
plt.show()
```

Heatmap: Trên Matplotlib

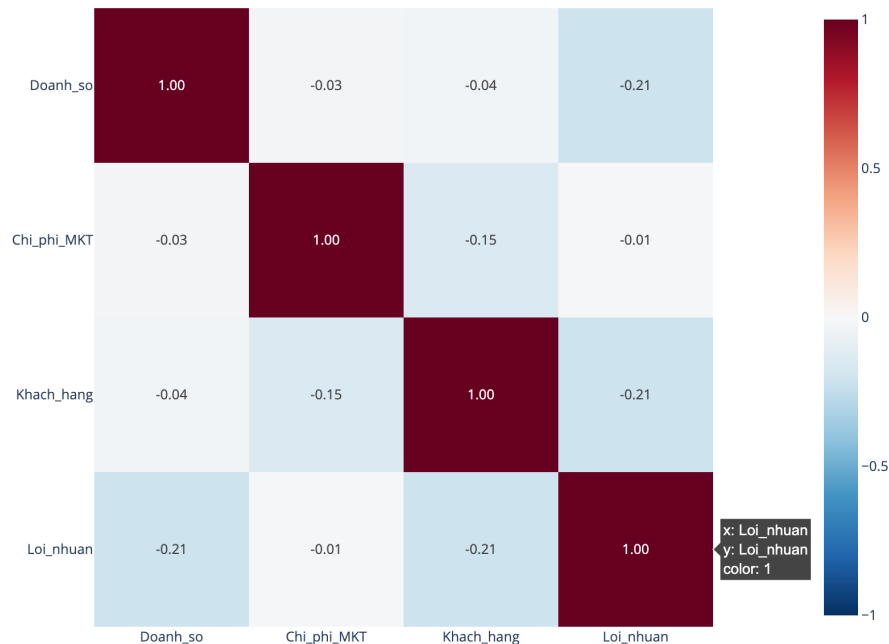
Một số tùy chỉnh nhanh:

Tham số	Giá trị gợi ý	Tác dụng
cmap	'Blues', 'coolwarm', 'magma', 'YlGnBu'	<i>Thay đổi dải màu thể hiện độ đậm nhạt của dữ liệu.</i>
interpolation	'nearest' (mặc định), 'gaussian', 'bilinear'	<i>Độ làm mịn/mờ đường viền giữa các ô dữ liệu.</i>
origin	'upper' (mặc định), 'lower'	<i>Đặt ô (0,0) ở góc trên cùng bên trái hay góc dưới cùng bên trái.</i>

Heatmap tương tác: Trên Plotly*

Vẽ ma trận tương quan (Correlation Matrix)

Ma trận Tương quan Tương tác



```
import numpy as np
import pandas as pd
import plotly.express as px
```

1. Khởi tạo dữ liệu Pandas DataFrame

```
np.random.seed(42)
df = pd.DataFrame({
    'Doanh_so': np.random.rand(100) * 100,
    'Chi_phi_MKT': np.random.rand(100) * 50,
    'Khách_hang': np.random.rand(100) * 200,
    'Lợi_nhuan': np.random.rand(100) * 30
})
```

2. Tính ma trận tương quan

```
corr = df.corr()
```

3. Vẽ Heatmap tương quan

```
fig = px.imshow(
    corr,
    text_auto='.2f',
    color_continuous_scale='RdBu_r',
    zmin=-1, zmax=1,
    title="Ma trận Tương quan Tương tác"
)
```

Làm tròn 2 chữ số thập phân
Màu đỏ-xanh thích hợp cho hệ số tương quan
Thang đo tương quan chuẩn từ -1 đến 1

```
fig.show()
```

Hệ số tương quan Pearson (r) đo lường mức độ và chiều hướng liên kết tuyến tính giữa hai biến định lượng X và Y .

Công thức tổng quát:

$$r = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^n (x_i - \bar{x})^2 \cdot \sum_{i=1}^n (y_i - \bar{y})^2}}$$

BÀI CŨ

...

Giải thích ký hiệu:

- n : Số lượng cặp mẫu dữ liệu.
- x_i, y_i : Giá trị quan sát thứ i của biến X và Y .
- $\bar{x}, \bar{y}$: Giá trị trung bình của tập X và Y ($\bar{x} = \frac{1}{n} \sum x_i, \bar{y} = \frac{1}{n} \sum y_i$).



Heatmap tương tác: Trên Plotly

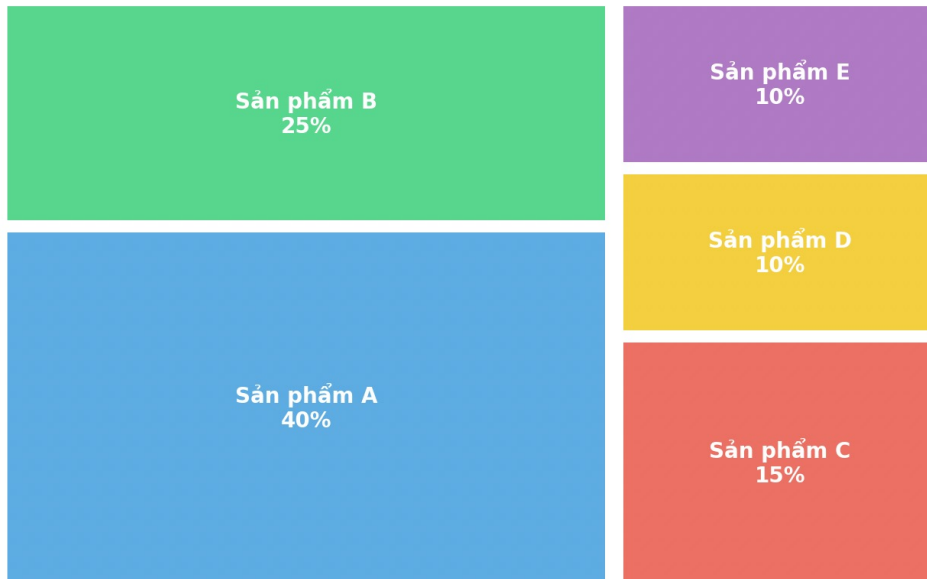
Một số tùy chỉnh nhanh:

Tham số	Giá trị gợi ý	Tác dụng
color_continuous_scale	'Viridis', 'Coolwarm', 'RdBu_r', 'Blues'	Đổi dải màu hiển thị mức độ dữ liệu.
text_auto	True hoặc '.2f'	Đổi cách hiển thị con số (số nguyên hoặc lấy 2 chữ số thập phân).
zmin / zmax	Ví dụ: zmin=-1, zmax=1	Cố định thang đo giới hạn nhỏ nhất/lớn nhất cho thanh màu.
aspect	'equal' hoặc 'auto'	Giữ ô vuông cân đối hoặc tự động dẫn theo kích thước màn hình.

Treemap : Trên Matplotlib | Squarify*

Vẽ Treemap cơ bản

Biểu đồ Treemap Thị phần Sản phẩm



```
import matplotlib.pyplot as plt
import squarify

# 1. Khởi tạo dữ liệu
sizes = [40, 25, 15, 10, 10]
labels = ['Sản phẩm A\n40%', 'Sản phẩm B\n25%',
          'Sản phẩm C\n15%', 'Sản phẩm D\n10%', 'Sản phẩm E\n10%']
colors = ['#3498db', '#2ecc71', '#e74c3c', '#f1c40f', '#9b59b6']

# 2. Vẽ Treemap
plt.figure(figsize=(8, 5))
squarify.plot(
    sizes=sizes,
    label=labels,
    color=colors,
    alpha=0.8,
    pad=True,
    text_kwargs={'fontsize': 10, 'color': 'white', 'weight': 'bold'}
)

# 3. Tắt trục tọa độ và hiển thị
plt.title('Biểu đồ Treemap Thị phần Sản phẩm', fontsize=14)
plt.axis('off')
plt.show()
```

**pip install squarify*



HCM-UTE

Treemap : Trên Matplotlib | Squarify*

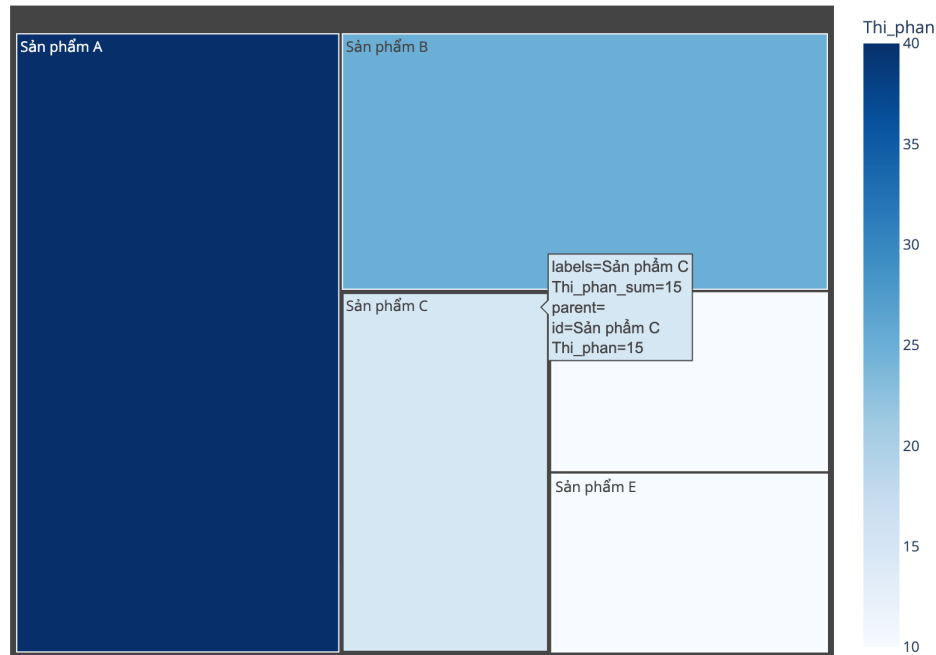
Một số tùy chỉnh nhanh trên Squarify.plot():

Tham số	Giá trị gợi ý	Tác dụng
sizes	Danh sách số thực/nguyên	<i>Kích thước (diện tích) tương quan của từng khối.</i>
label	Danh sách chuỗi ký tự	<i>Nhãn chữ hiển thị trên từng khối.</i>
color	Danh sách mã màu Hex	<i>Tùy chỉnh màu sắc riêng cho từng khối hình chữ nhật.</i>
pad	True / False	<i>Tạo khoảng phân cách (khe hở) giữa các hình chữ nhật giúp dễ nhìn hơn.</i>
alpha	0.1 đến 1.0	<i>Độ mờ / trong suốt của màu nền.</i>
text_kw	dict(fontsize=12, color='black')	<i>Tùy chỉnh font chữ, cỡ chữ, màu chữ cho nhãn dữ liệu.</i>

Treemap tương tác: Trên Plotly*

Treemap tương tác đơn giản 1 cấp

Biểu đồ Treemap Tương tác Đơn giản



```
import pandas as pd
import plotly.express as px
```

1. Khởi tạo dữ liệu mẫu

```
df = pd.DataFrame({
    'San_pham': ['Sản phẩm A', 'Sản phẩm B',
                 'Sản phẩm C', 'Sản phẩm D', 'Sản phẩm E'],
    'Thi_phan': [40, 25, 15, 10, 10]
})
```

2. Vẽ Treemap tương tác

```
fig = px.treemap(
    df,
    path=['San_pham'],          # Xác định nhãn các ô
    values='Thi_phan',          # Tỷ lệ diện tích các ô
    color='Thi_phan',           # Đổi màu tự động theo thị phần
    color_continuous_scale='Blues', # Bảng màu xanh
    title="Biểu đồ Treemap Tương tác Đơn giản"
)

fig.show()
```

Treemap tương tác: Trên Plotly*

Treemap tương tác Phân cấp

Treemap Phân cấp (Nhấp vào ô 'Điện thoại' hoặc 'Laptop' để mở chi tiết)



```
import pandas as pd
import plotly.express as px
```

1. Dữ liệu phân cấp: Danh mục -> Sản phẩm

```
df = pd.DataFrame({
    'Danh_muc': ['Điện thoại', 'Điện thoại',
                'Laptop', 'Laptop', 'Thời trang', 'Thời trang'],
    'San_pham': ['iPhone 15', 'Samsung S24', 'MacBook Air',
                'Dell XPS', 'Áo sơ mi', 'Quần Jeans'],
    'Doanh_so': [450, 300, 500, 350, 200, 150]
})
```

2. Vẽ Treemap phân cấp

```
fig = px.treemap(
    df,
    path=['Danh_muc', 'San_pham'], # Cấp 1: Danh mục -> Cấp 2: Sản phẩm
    values='Doanh_so',
    color='Doanh_so',
    color_continuous_scale='Viridis',
    title="Treemap Phân cấp (Nhấp vào ô 'Điện thoại' hoặc 'Laptop' để mở chi tiết)"
)
```

Cấu hình hiển thị chữ lớn, rõ ràng

```
fig.update_traces(root_color="lightgrey")
fig.show()
```



Treemap tương tác: Trên Plotly

Một số tùy chỉnh nhanh:

Tham số	Giá trị gợi ý	Tác dụng
path	['Cap_1', 'Cap_2']	<i>Danh sách các cột thể hiện độ phân cấp từ lớn đến nhỏ (Cha -> Con).</i>
values	Tên cột số liệu (như Doanh_so)	<i>Xác định kích thước diện tích khối hình chữ nhật.</i>
color	Tên cột số liệu hoặc phân loại	<i>Tô màu khối theo giá trị hoặc nhóm.</i>
color_continuous_scale	'Viridis', 'RdBu', 'Spectral'	<i>Chọn bảng màu chuyển sắc liên tục.</i>
hover_data	['Loi_nhuan', 'So_luong']	<i>Bổ sung thông tin muốn hiển thị thêm khi di chuột vào khối.</i>

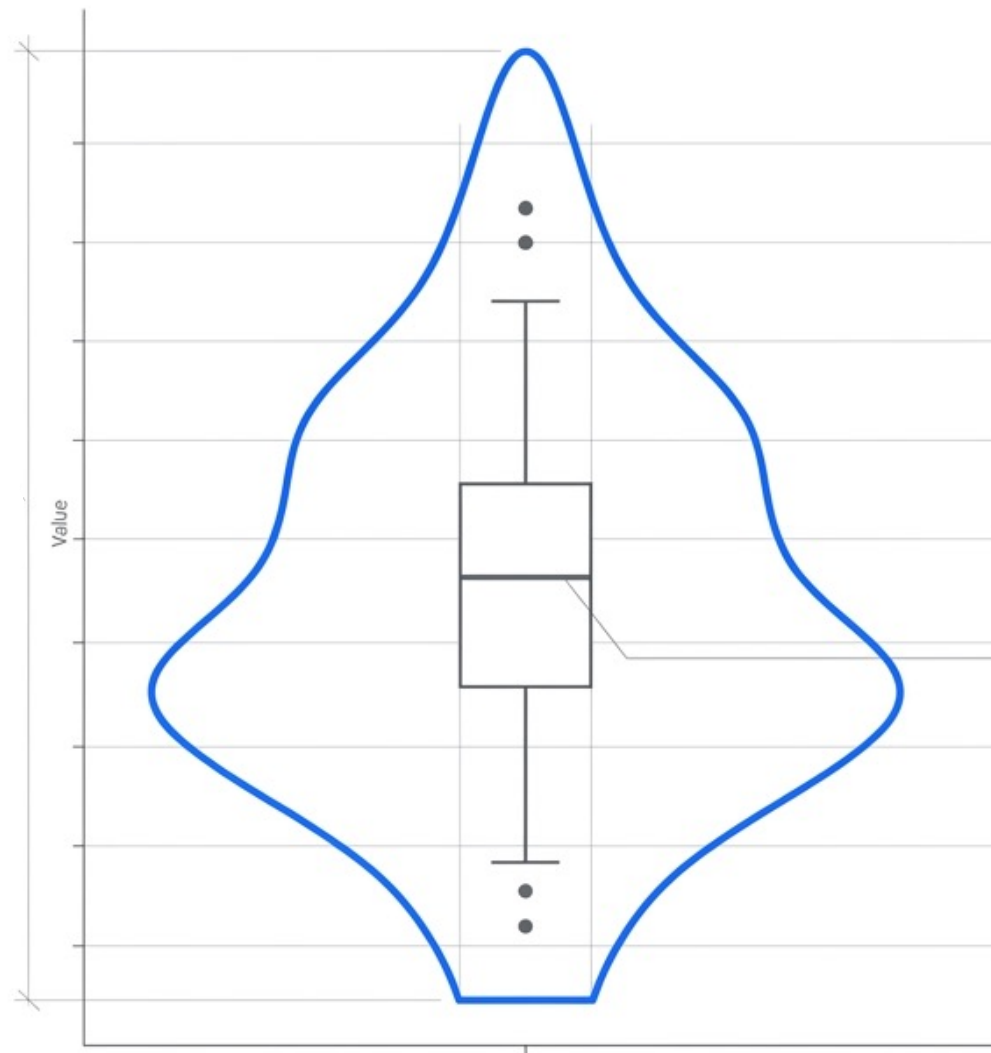


HCM-UTE

Violin Plot: Sự kết hợp hoàn hảo (Box + KDE)

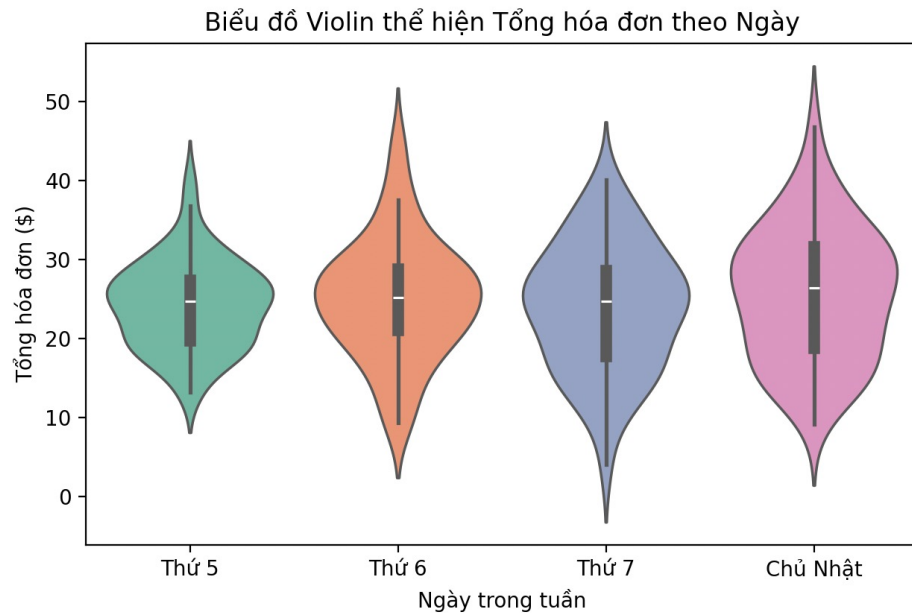
- Box Plot cho thấy các điểm mốc thống kê (tứ phân vị), nhưng giấu đi hình dạng thực sự của phân phối dữ liệu.
- Biểu đồ Violin giải quyết vấn đề này bằng cách nhúng một đường cong mật độ (density curve) đối xứng vào xung quanh Box Plot.

Nó trả lời đồng thời hai câu hỏi: "Dữ liệu tập trung ở đâu?" và "Hình dáng phân phối của nó như thế nào?".



Violin plot : Trên Seaborn*

Vẽ Violin plot cơ bản



```
import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd
import numpy as np
```

1. Tự khởi tạo dữ liệu giả lập

```
np.random.seed(42)
df = pd.DataFrame({
    'day': ['Thứ 5', 'Thứ 6', 'Thứ 7', 'Chủ Nhật'] * 50,
    'total_bill': np.random.normal(loc=25, scale=8, size=200)
})
```

2. Vẽ Violin Plot

```
plt.figure(figsize=(7, 4))
sns.violinplot(data=df, x="day", y="total_bill", palette="Set2", inner="box")
```

```
plt.title("Biểu đồ Violin thể hiện Tổng hóa đơn theo Ngày")
plt.xlabel("Ngày trong tuần")
plt.ylabel("Tổng hóa đơn ($)")
plt.show()
```

**pip install seaborn*

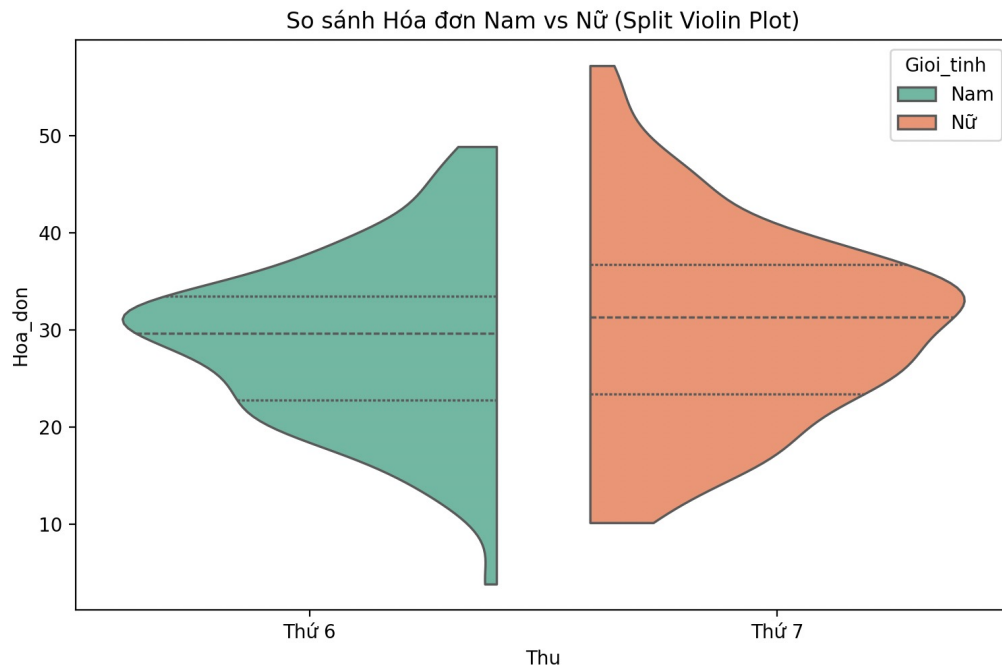
Violin plot : Trên Seaborn*

Một số tùy chỉnh nhanh:

Tham số	Giá trị gợi ý	Tác dụng
hue	Tên cột phân loại (VD: 'sex')	<i>Chia nhỏ dữ liệu thành các nhóm con để so sánh trong cùng 1 trục.</i>
split	True	<i>Chia đôi thân violin thành 2 nửa trái/phải (dùng khi có hue) giúp tiết kiệm không gian.</i>
inner	'box', 'quartiles', 'stick', None	<i>Kiểu hiển thị lõi: 'box' (hộp), 'quartiles' (3 đường tứ phân vị), 'stick' (vạch từng điểm dữ liệu).</i>
cut	0	<i>Cắt bỏ phần đuôi dư thừa, giới hạn đường cong đúng trong khoảng min-max thực tế của dữ liệu.</i>
bw_adjust	0.5 đến 2.0	<i>Điều chỉnh độ mượt đường cong KDE (số càng nhỏ đường càng nhấp nhô chi tiết, số lớn đường càng mượt).</i>
orient	'h' hoặc 'v'	<i>Xoay ngang ('h') hoặc dựng đứng ('v') biểu đồ.</i>

Violin plot : Trên Seaborn*

Vẽ Violin plot tùy chỉnh



**pip install seaborn*

```
import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd
import numpy as np
```

```
# Khởi tạo dữ liệu mẫu
```

```
np.random.seed(42)
df = pd.DataFrame({
    'Thu': ['Thứ 6', 'Thứ 7'] * 100,
    'Gioi_tinh': ['Nam', 'Nữ'] * 100,
    'Hoa_don': np.random.normal(loc=30, scale=10, size=200)
})
```

```
plt.figure(figsize=(8, 4.5))
sns.violinplot(
    data=df,
    x='Thu',
    y='Hoa_don',
    hue='Gioi_tinh',
    split=True,
    inner='quartiles',
    cut=0,
    palette='Set2',
    bw_adjust=0.8
)
```

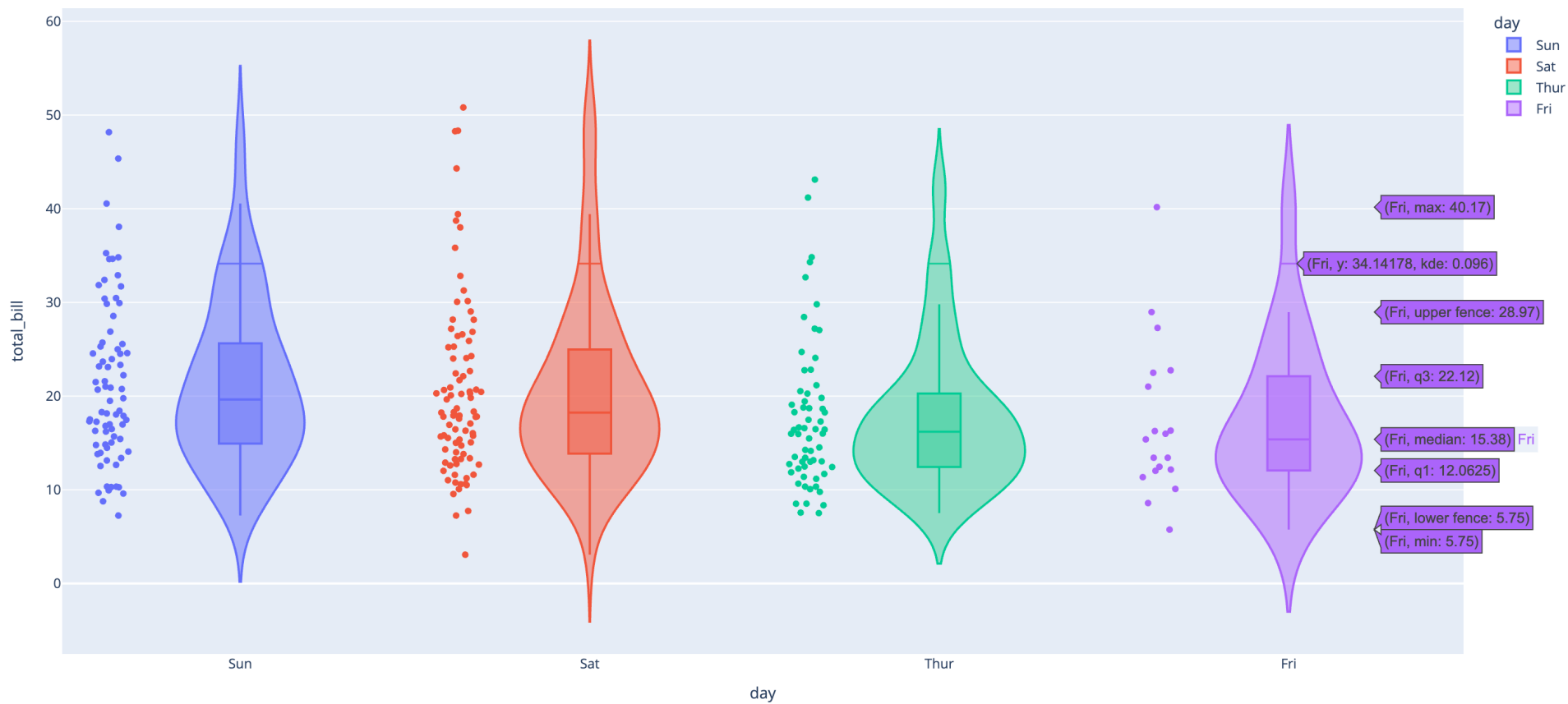
```
# Gộp 2 nửa Nam/Nữ vào cùng 1 violin
# Hiện 3 đường gạch tứ phân vị bên trong
# Không vẽ phần đuôi giả lập ngoài min/max
# Bảng màu
# Chỉnh độ mượt đường cong
```

```
plt.title('So sánh Hóa đơn Nam vs Nữ (Split Violin Plot)')
plt.show()
```

Violin plot tương tác: Trên Plotly*

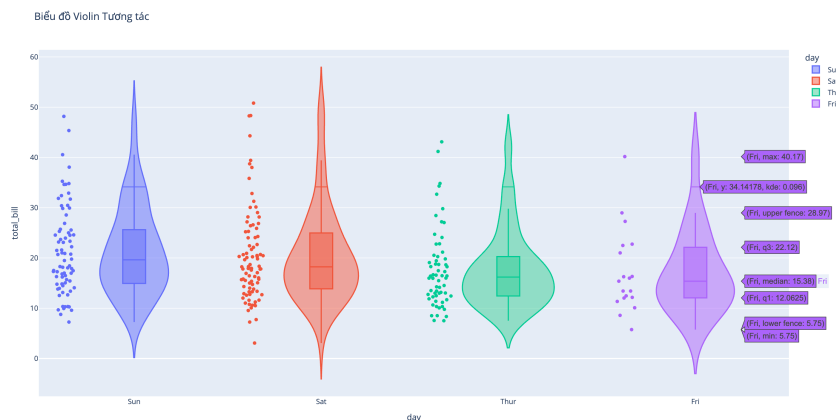
Violin tương tác đơn giản

Biểu đồ Violin Tương tác



Violin plot tương tác: Trên Plotly*

Violin tương tác đơn giản



```
import plotly.express as px
```

```
df = px.data.tips()
```

```
fig = px.violin(  
    df,  
    y="total_bill",  
    x="day",  
    color="day",  
    box=True, # Hiển thị thêm khung Box Plot  
    points="all", # Hiển thị tất cả các điểm  
    title="Biểu đồ Violin Tương tác"  
)
```

```
fig.show()
```

Violin plot tương tác: Trên Plotly

Một số tùy chỉnh nhanh:

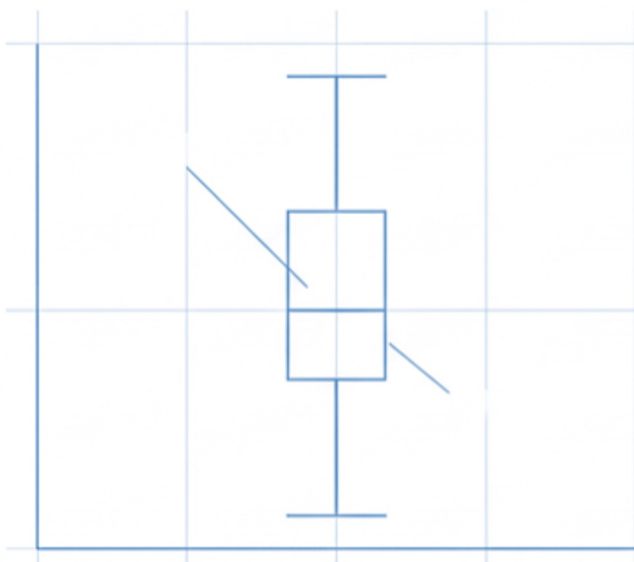
Tham số	Giá trị gợi ý	Tác dụng
box	True	Vẽ thêm một chiếc hộp Box Plot thu nhỏ ngay chính giữa lõi violin.
points	'all', 'outliers', False	Hiển thị tất cả các điểm dữ liệu thực tế rải rác bên cạnh thân violin.
violinmode	'group', 'overlay'	Đặt các nhóm violin nằm cạnh nhau hay đè mờ lên nhau.
hover_data	['Cot_1', 'Cot_2']	Bổ sung thêm các thông tin phụ khi rê chuột vào điểm dữ liệu.



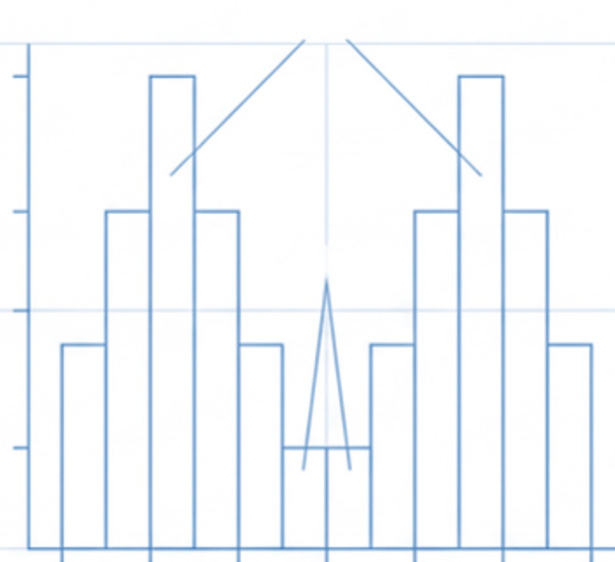
HCM-UTE

Ba góc nhìn khác nhau trên cùng một dữ liệu

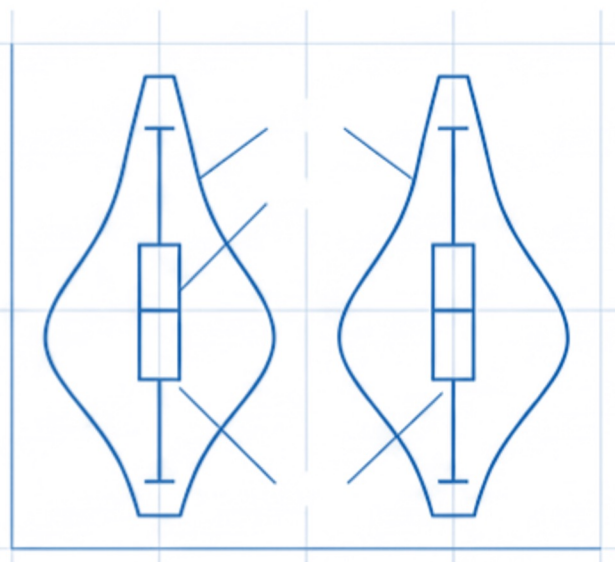
Giả sử chúng ta có một tập dữ liệu với hai nhóm tuổi tách biệt (bimodal distribution).



Góc nhìn Box Plot: Chỉ thấy dữ liệu đối xứng, không phát hiện được hai đỉnh. (Dễ gây hiểu lầm).



Góc nhìn Histogram: Nhìn thấy rõ hai đỉnh (tần suất), nhưng thiếu các mốc tứ phân vị rõ ràng.



Góc nhìn Violin Plot: Sự kết hợp hoàn chỉnh—thấy cả cấu trúc thống kê lẫn mật độ phân phối.

Không có biểu đồ nào là hoàn hảo tuyệt đối! Sự lựa chọn phụ thuộc vào dữ liệu và câu hỏi phân tích từ bài toán

Kết hợp biểu đồ: Sức mạnh của Dual - Axis

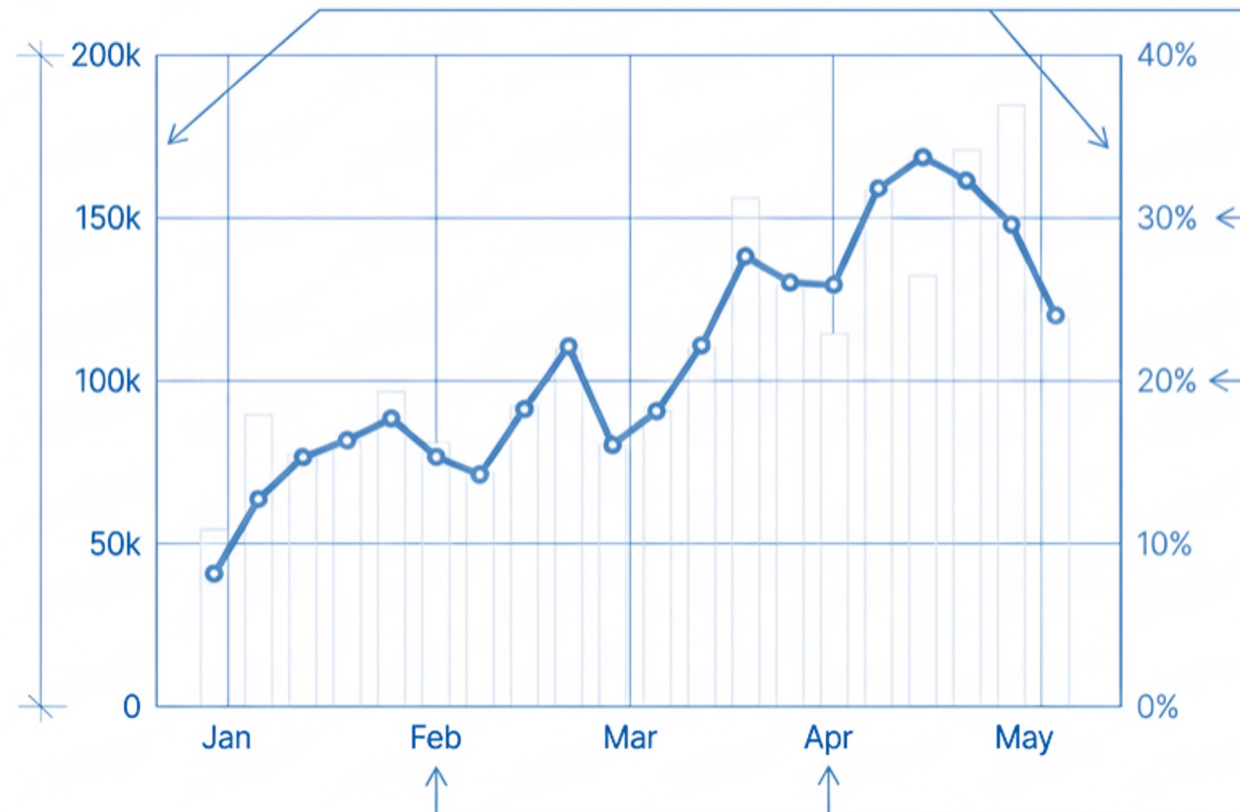
Việc kết hợp Line & Bar trên cùng một không gian (Dual-axis charts) cho phép so sánh hai biến số có thang đo hoặc đơn vị đo lường hoàn toàn khác nhau.

Cơ chế hoạt động:

- Trục Y chính (Bên trái): Dành cho Biểu đồ Cột (thường đo lường số lượng/khối lượng tuyệt đối).
- Trục Y phụ (Bên phải): Dành cho Biểu đồ Đường (thường đo lường tỷ lệ phần trăm hoặc xu hướng).

Mục đích:

- Tìm kiếm mối tương quan giữa khối lượng và hiệu suất trong cùng một khung thời gian.



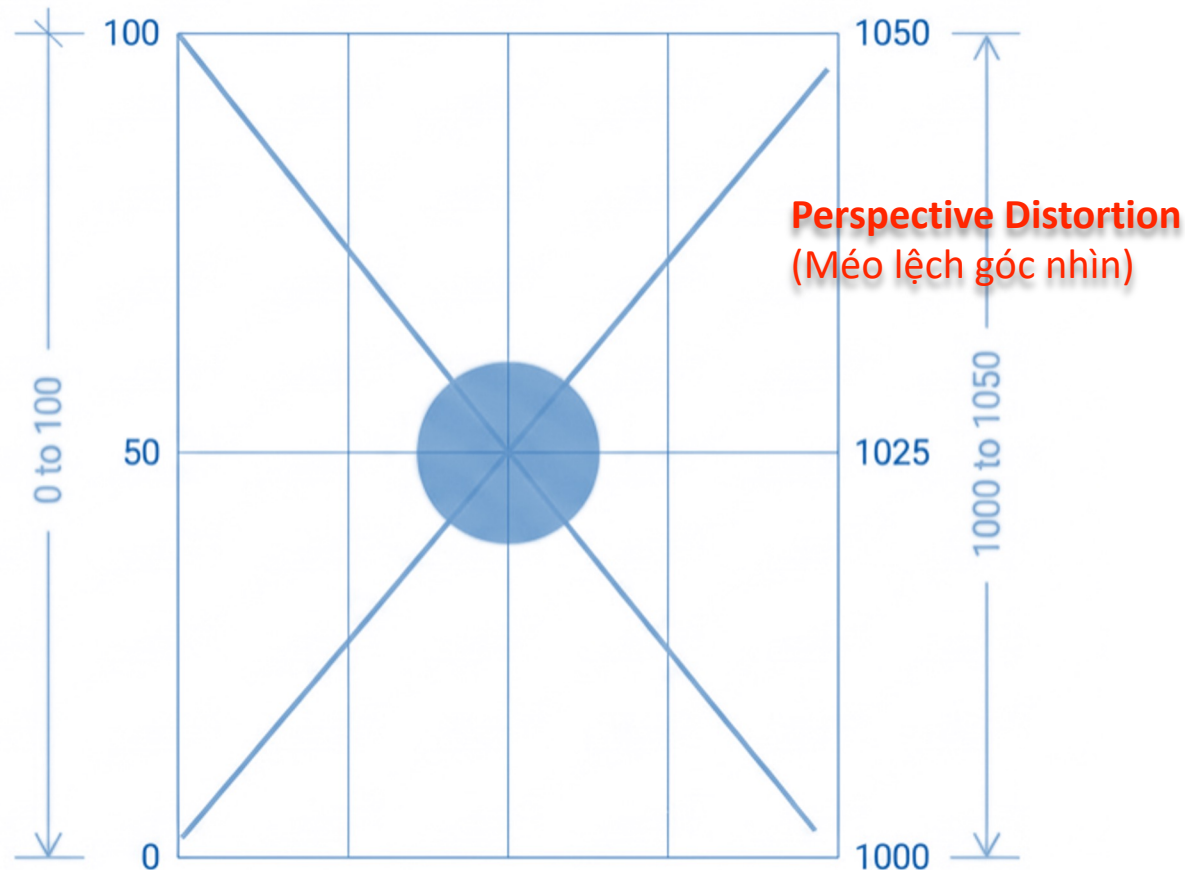
Lưu ý cần thiết khi dùng biểu đồ hai trục

Biểu đồ hai trục rất mạnh mẽ, nhưng cũng là công cụ dễ gây hiểu lầm nhất nếu không cẩn thận.

Điểm mù thị giác (Điểm giao cắt ảo):

- Bộ não con người tự động tìm kiếm ý nghĩa khi hai đường giao nhau.
- Tuy nhiên, nếu hai trục không được đồng bộ hóa (synchronize) hoặc sử dụng thang đo lệch nhau, điểm giao cắt này hoàn toàn vô nghĩa.

Giải pháp: Luôn ghi chú rõ ràng trục nào thuộc về đường nào, và cân nhắc sử dụng hai biểu đồ xếp chồng lên nhau (Small multiples) thay vì ép chúng chung một khung hình.





HCM-UTE

HO CHI MINH CITY

UNIVERSITY OF TECHNOLOGY AND ENGINEERING

01, Vo Van Ngan street, Thu Duc ward, Ho Chi Minh City, Vietnam

hcmute.edu.vn

Trực quan hóa dữ liệu địa lý thông qua bản đồ

Khi dữ liệu mang theo vĩ độ, kinh độ hoặc ranh giới hành chính, Bản đồ (Maps) là công cụ duy nhất để khám phá ngữ cảnh không gian.

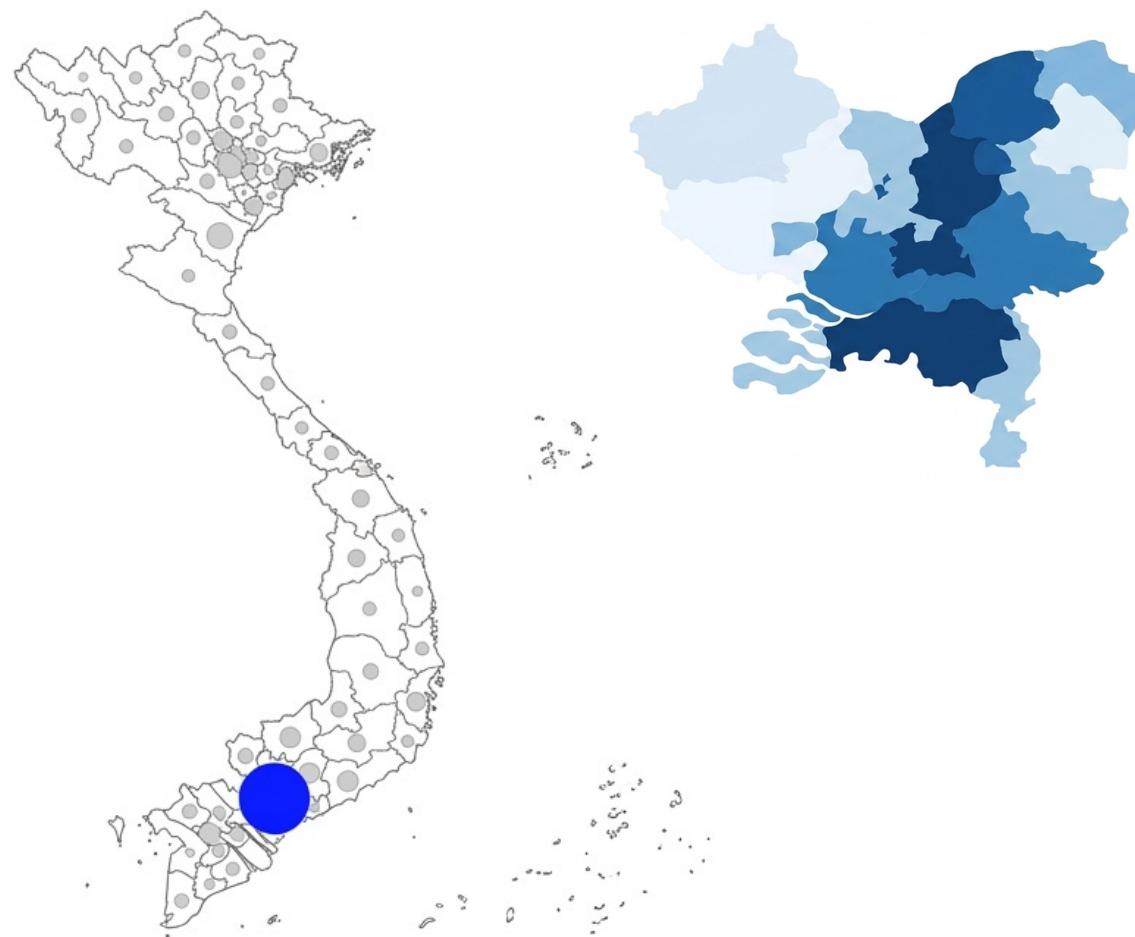
Hai phương pháp phổ biến:

Bản đồ ký hiệu tỷ lệ (Proportional Symbol Map):

Sử dụng kích thước của các chấm tròn (như biểu đồ) đặt trên các tọa độ để thể hiện độ lớn của giá trị. (Phù hợp cho dữ liệu điểm cụ thể).

Bản đồ sắc ký (Choropleth Map):

Tô màu các khu vực hành chính (tỉnh/thành phố) theo một thang màu gradient. (Phù hợp cho dữ liệu tỷ lệ/mật độ, ví dụ: dân số/km²).





HCM-UTE

HO CHI MINH CITY

UNIVERSITY OF TECHNOLOGY AND ENGINEERING

01, Vo Van Ngan street, Thu Duc ward, Ho Chi Minh City, Vietnam

hcmute.edu.vn

Proportional Symbol map cơ bản: Trên Matplotlib | GeoPandas*

```
import geopandas as gpd
import matplotlib.pyplot as plt
import pandas as pd
```

1. Dữ liệu các thành phố

```
data = {
    'City': ['Hà Nội', 'Hải Phòng', 'Đà Nẵng',
            'TP. Hồ Chí Minh', 'Cần Thơ'],
    'Lat': [21.0285, 20.8449, 16.0544, 10.8231, 10.0452],
    'Lon': [105.8542, 106.6881, 108.2022, 106.6297, 105.7469],
    'Dan_so': [8.5, 2.1, 1.2, 9.3, 1.3]
}
df = pd.DataFrame(data)
```

2. Link GeoJSON Việt Nam

```
url_vietnam = "vn_geo.json"
vietnam_map = gpd.read_file(url_vietnam)
```

3. Vẽ bản đồ

```
fig, ax = plt.subplots(figsize=(8, 12))
vietnam_map.plot(ax=ax, color='#e0e0e0',
                 edgecolor='white', linewidth=0.8)
```

4. Vẽ bong bóng dữ liệu

```
ax.scatter(
    df['Lon'],
    df['Lat'],
    s=df['Dan_so'] * 100,
    c=df['Dan_so'],
    cmap='Reds',
    alpha=0.6,
    edgecolors='black',
    linewidth=1.2
)
```

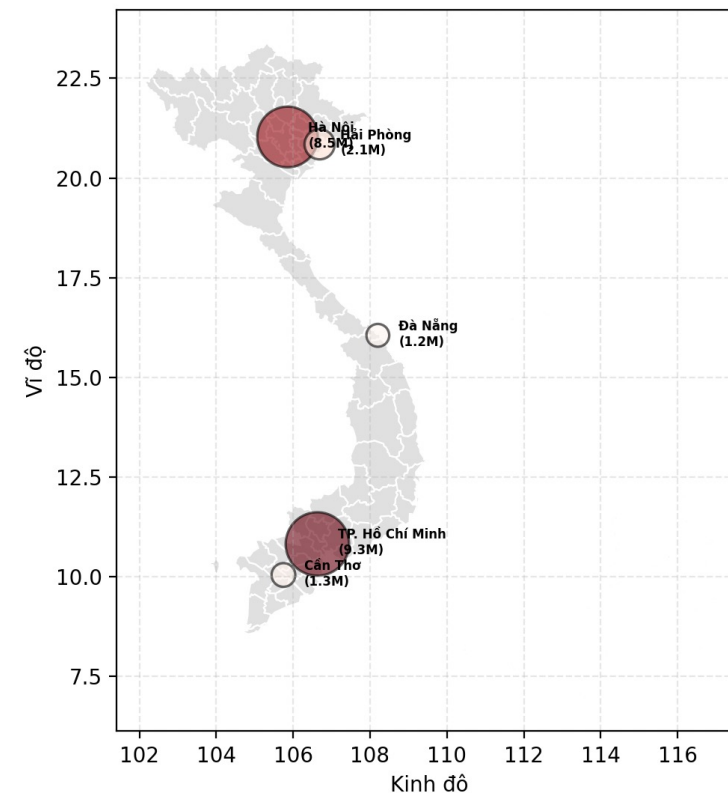
5. Gán nhãn tên thành phố

```
for idx, row in df.iterrows():
    ax.annotate(
        f"{row['City']}\n({row['Dan_so']}M)",
        (row['Lon'], row['Lat']),
        textcoords="offset points",
        xytext=(10, -5),
        fontsize=6,
        weight='bold'
    )

plt.title('Bản đồ Dân số các TP lớn tại Việt Nam',
         fontsize=11, weight='bold', pad=10)
plt.xlabel('Kinh độ')
plt.ylabel('Vĩ độ')
plt.grid(True, linestyle='--', alpha=0.3)

plt.show()
```

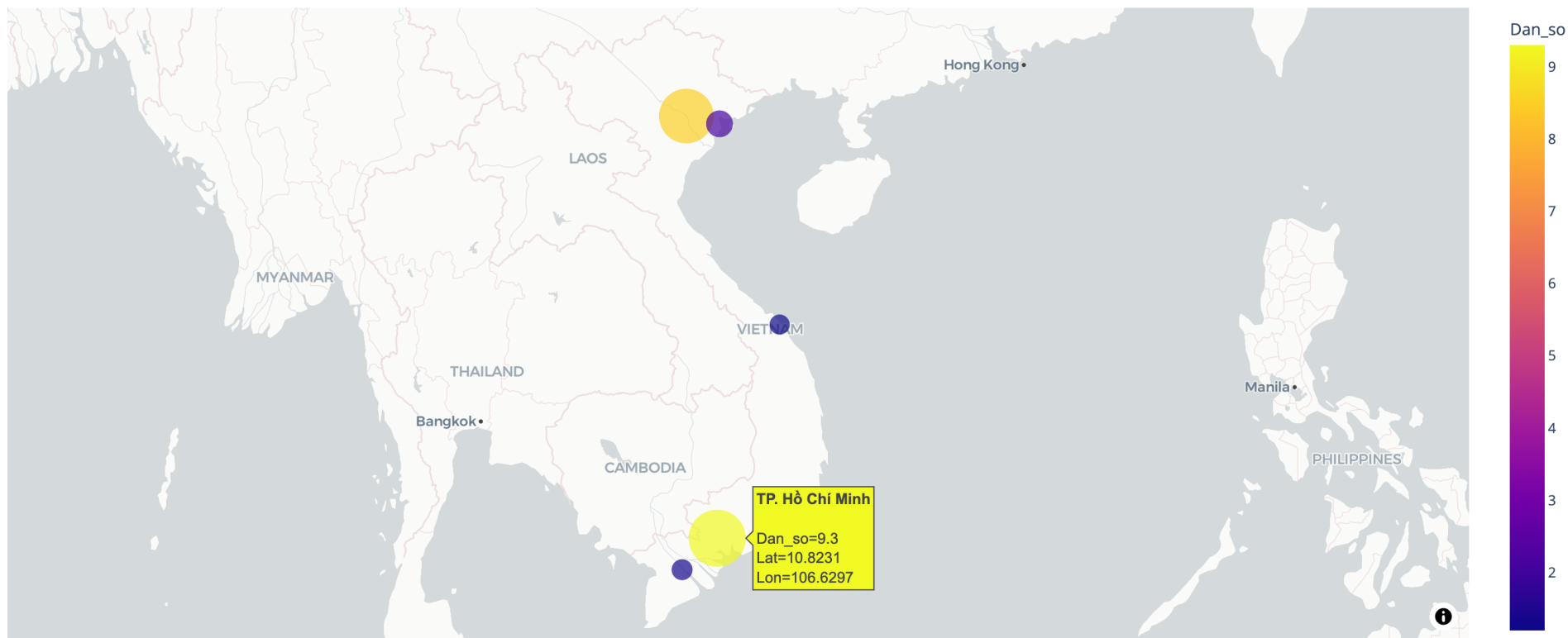
Bản đồ Dân số các TP lớn tại Việt Nam



*pip install geopandas matplotlib pandas

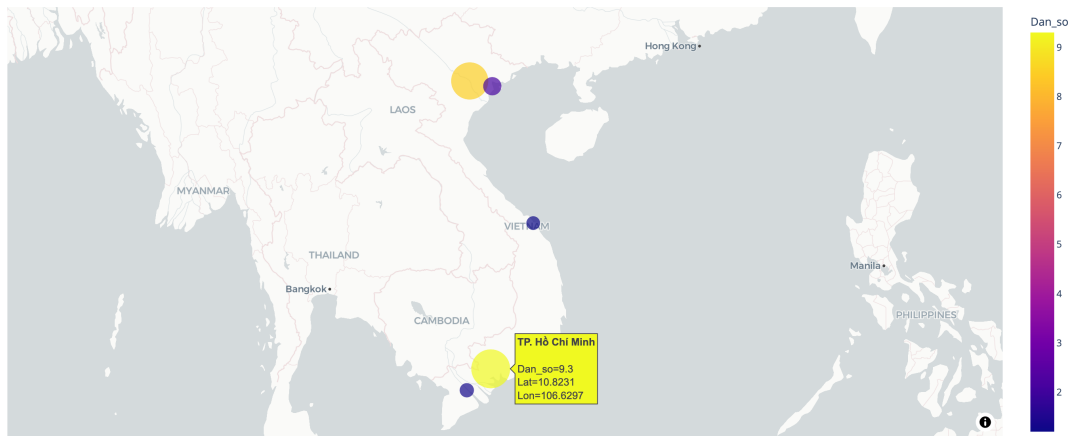
Proportional Symbol map cơ bản: Trên Plotly*

Bản đồ Dân số Tương tác các Thành phố Việt Nam



Proportional Symbol map cơ bản: Trên Plotly*

Bản đồ Dân số Tương tác các Thành phố Việt Nam



```
import pandas as pd
import plotly.express as px
```

1. Khởi tạo dữ liệu

```
data = {
    'City': ['Hà Nội', 'Hải Phòng', 'Đà Nẵng', 'TP. Hồ Chí Minh', 'Cần Thơ'],
    'Lat': [21.0285, 20.8449, 16.0544, 10.8231, 10.0452],
    'Lon': [105.8542, 106.6881, 108.2022, 106.6297, 105.7469],
    'Dan_so': [8.5, 2.1, 1.2, 9.3, 1.3]
}
df = pd.DataFrame(data)
```

2. Vẽ Proportional Symbol Map tương tác

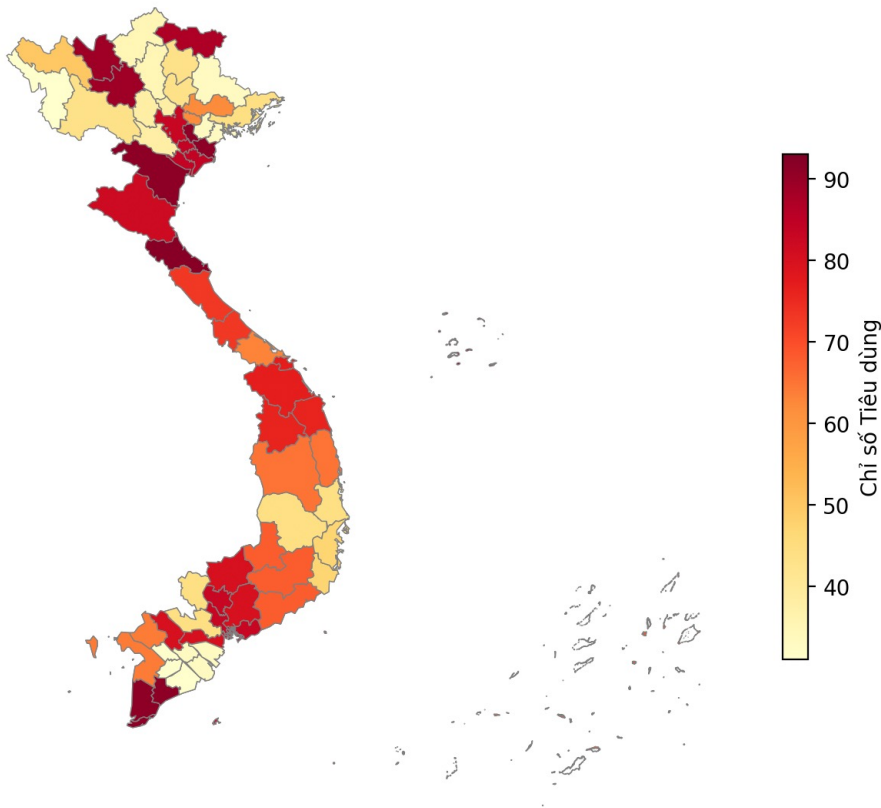
```
fig = px.scatter_map(
    df,
    lat="Lat",
    lon="Lon",
    size="Dan_so",
    color="Dan_so",
    hover_name="City",
    size_max=35,
    zoom=4.8,
    center={"lat": 16.0, "lon": 106.5},
    map_style="carto-positron",
    title="Bản đồ Dân số Tương tác các Thành phố Việt Nam"
)

fig.show()
```

Kích thước bong bóng theo dân số
Màu sắc theo dân số
Hiện thị tên thành phố khi di chuột
Kích thước vòng tròn tối đa
Mức zoom vừa vặn bản đồ Việt Nam
Căn trung tâm Việt Nam
Nền bản đồ sạch, đẹp

Choropleth Map cơ bản: Trên Matplotlib | GeoPandas*

Bản đồ Choropleth Tỉnh/Thành Việt Nam



```
import geopandas as gpd
import matplotlib.pyplot as plt
import pandas as pd
import numpy as np

# 1. Đọc trực tiếp GeoJSON (SV tải trên utexlms Thầy Trí gửi)
url_tinh_thanh = "vn_geo.json"
vietnam_map = gpd.read_file(url_tinh_thanh)

# 2. Tạo dữ liệu giả lập chỉ số kinh tế/dân số cho 63 tỉnh/thành
name_col = 'Name' if 'Name' in vietnam_map.columns else 'name'

np.random.seed(42)
data_df = pd.DataFrame({
    name_col: vietnam_map[name_col],
    'Chi_so': np.random.randint(30, 100, size=len(vietnam_map))
})

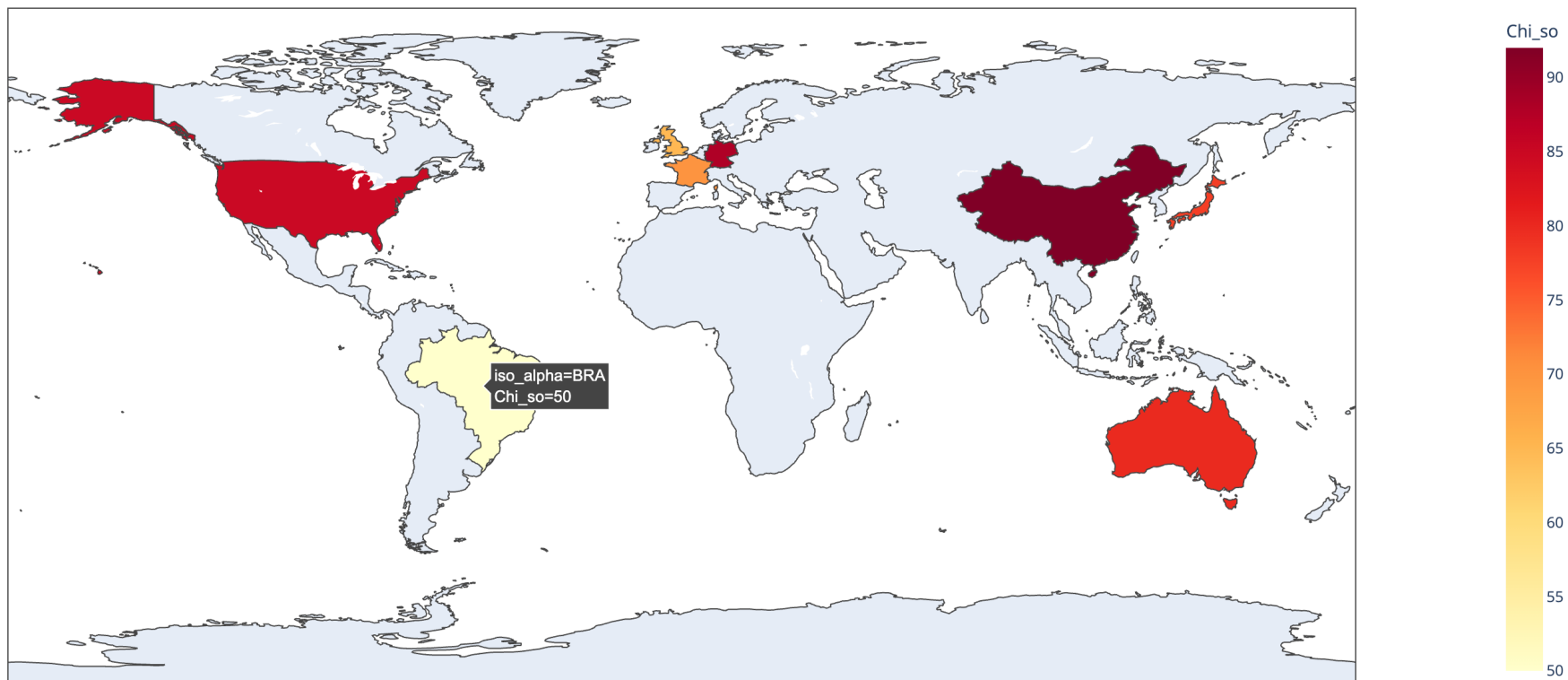
# 3. Kết hợp dữ liệu địa lý và chỉ số
merged_map = vietnam_map.merge(data_df, on=name_col)

# 4. Vẽ Choropleth Map
fig, ax = plt.subplots(figsize=(8, 12))
merged_map.plot(
    column='Chi_so',
    cmap='YlOrRd',
    linewidth=0.5,
    ax=ax,
    edgecolor='0.5',
    legend=True,
    legend_kwds={'label': "Chỉ số Tiêu dùng", 'shrink': 0.5}
)

plt.title('Bản đồ Choropleth Tỉnh/Thành Việt Nam', fontsize=13, weight='bold')
plt.axis('off')
plt.show()
```

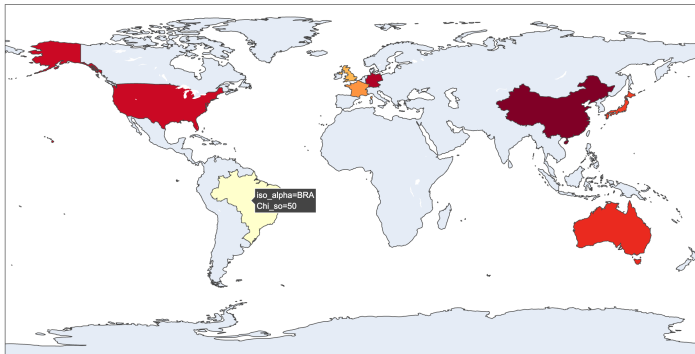
Choropleth Map cơ bản: Trên Plotly

Chỉ số Tiêu dùng theo Quốc gia (Ví dụ minh họa)



Choropleth Map cơ bản: Trên Plotly

Chỉ số Tiêu dùng theo Quốc gia (Ví dụ minh họa)



```
import plotly.express as px
import pandas as pd
import numpy as np
```

1. Tạo dữ liệu giả lập cho một số quốc gia

```
df = pd.DataFrame({
    "iso_alpha": ["USA", "CHN", "JPN", "DEU", "FRA", "GBR", "BRA", "AUS"],
    "Chi_so": [85, 92, 78, 88, 70, 65, 50, 80]
})
```

2. Vẽ bản đồ Choropleth

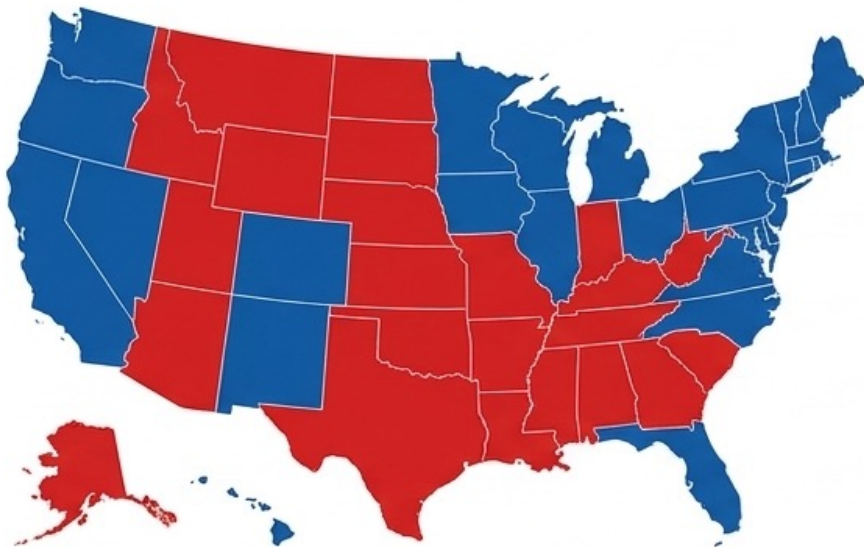
```
fig = px.choropleth(
    df,
    locations="iso_alpha",
    color="Chi_so",
    color_continuous_scale="YlOrRd",
    title="Chỉ số Tiêu dùng theo Quốc gia (Ví dụ minh họa)"
)
```

```
fig.show()
```

Thiết kế bản đồ: Đảm bảo tính hiệu quả

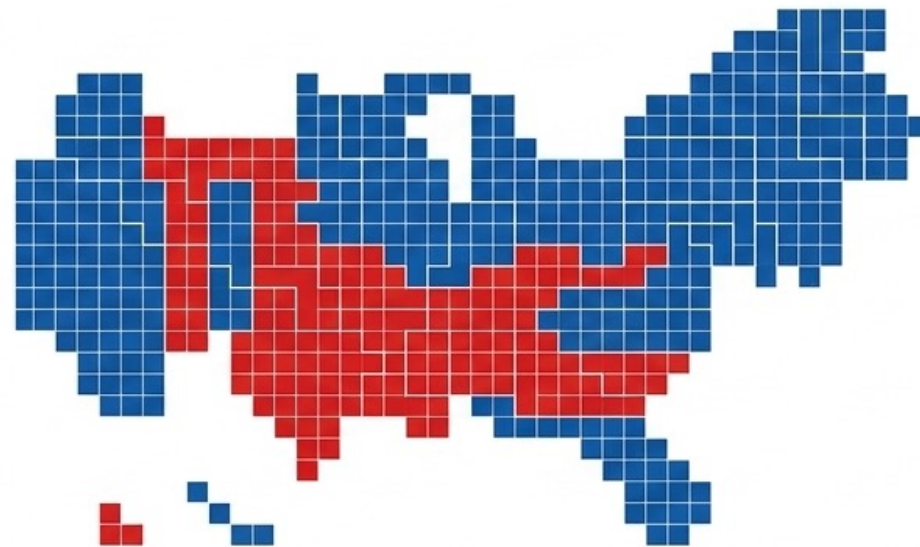
Case study: Bản đồ Bầu cử Mỹ - The New York Times (từ Beautiful Visualization)

Nhấn mạnh diện tích vật lý



5 bang (đỏ) rộng lớn trông có vẻ áp đảo hoàn toàn bề mặt, nhưng thực chất chúng chỉ đóng góp tổng cộng 16 phiếu đại cử tri. Sự chính xác về địa lý gây hiểu lầm nghiêm trọng về sức mạnh dữ liệu.

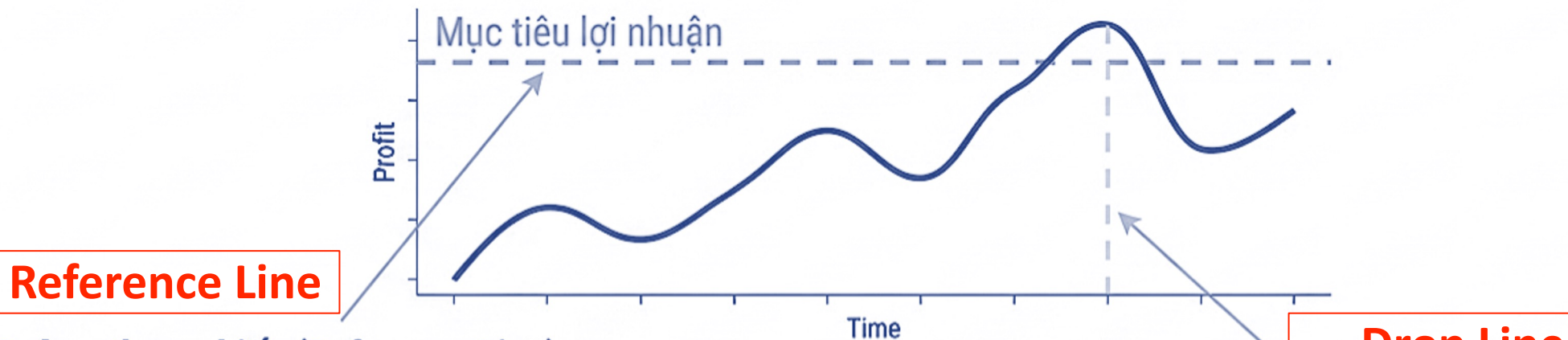
Ưu tiên dữ liệu cốt lõi



Hình dáng địa lý bị bóp méo, diện tích được vẽ lại dựa trên số phiếu bầu. Nó phản ánh chính xác cán cân quyền lực thực sự (16 phiếu của 5 bang lớn vs 15 phiếu của 1 bang New Jersey nhỏ bé).

Bài học: Một bản đồ trực quan xuất sắc phải phục vụ thông điệp của dữ liệu, không phải phục vụ hình học thuần túy.

Tạo ngữ cảnh cho dữ liệu



Đường tham chiếu (Reference Line):

- Bản chất: Là một vạch đích cố định, giá trị trung bình toàn ngành, hoặc giới hạn cảnh báo rủi ro.
- Giá trị tương tác: Giúp người xem không phải tự nhầm tính. Nó trả lời ngay lập tức câu hỏi: "So với tiêu chuẩn, kết quả hiện tại là tốt hay xấu?"

Đường thả xuống (Drop Line):

- Bản chất: Là đường dóng từ một điểm dữ liệu lơ lửng xuống thẳng các trục tọa độ.
- Giá trị tương tác: Cứu cánh cho các biểu đồ phức tạp có không gian rộng. Giúp mắt người xem dễ dàng định vị chính xác giá trị trục X/Y mà không bị chệch hướng nhìn.



Tạo ngữ cảnh cho dữ liệu

Reference Line: Luôn ưu tiên dùng `axhline/add_hline` thay vì vẽ đường thẳng thường để đồ thị tự co giãn linh hoạt khi thay đổi dải dữ liệu.

Drop Line tĩnh vs. Động: Matplotlib phù hợp cho báo cáo in ấn/slide tĩnh (dùng `vlines`), trong khi Plotly có lợi thế vượt trội nhờ `showspikes` giúp giao diện tương tác gọn gàng, chỉ hiện đường dóng khi cần quan sát.

Đây là nội dung tự học có hướng dẫn, Sinh viên tự nghiên cứu thêm phần này

Sức mạnh của tương tác: Công cụ chú giải (Tooltips)

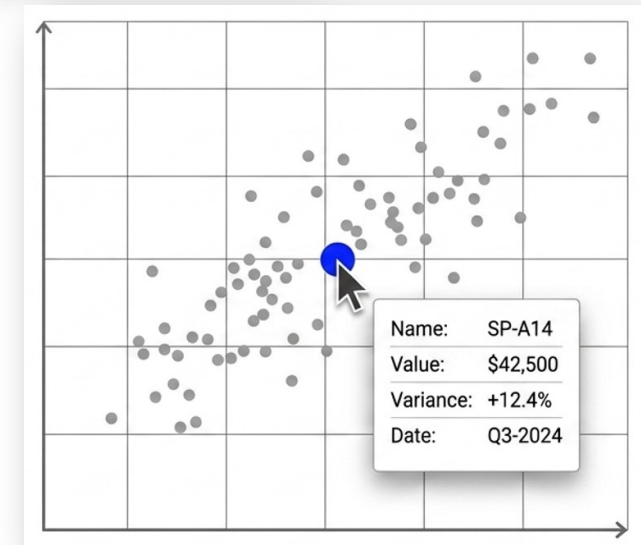
Giữ cho biểu đồ tinh gọn bề ngoài, nhưng chứa đựng kho tàng dữ liệu bên trong.

Khái niệm: Hộp thông tin bổ sung xuất hiện tức thời khi người dùng di chuột (hover) hoặc chạm vào một điểm dữ liệu trên màn hình.

Vai trò cốt lõi trong Dashboard hiện đại:

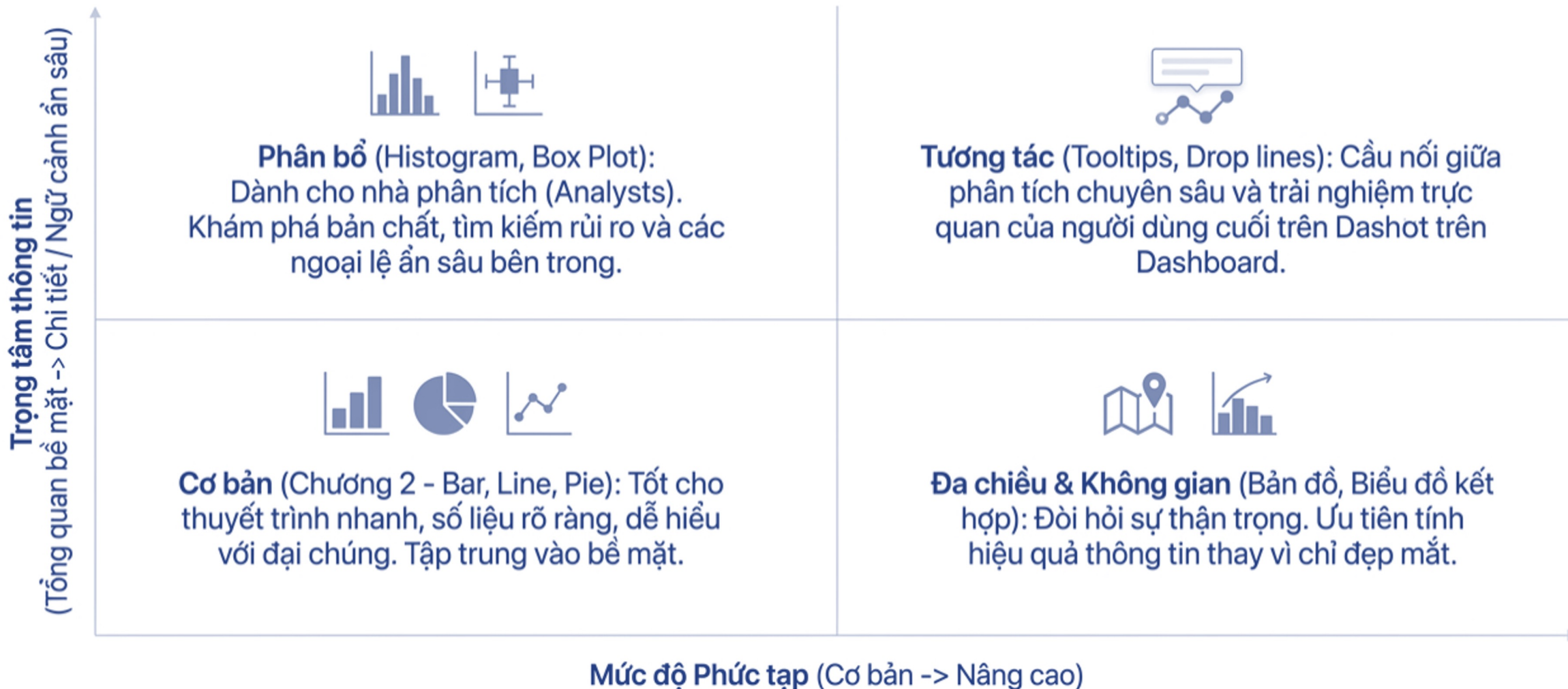
- Giảm nhiễu (Reduce Noise): Tuân thủ triết lý tối giản, không bắt khán giả đọc mọi con số trên màn hình tĩnh.
- Cung cấp ngữ cảnh ngầm (Provide Context): Chứa các trường dữ liệu phụ trợ không thể nhét vừa trục X và Y (ví dụ: nguyên nhân sụt giảm, tên nhân sự phụ trách).

Mẫu chốt thiết kế: Tooltip khuyến khích người dùng cuối chủ động tự **khám phá** (Explore) thay vì bị động tiếp nhận.



Đây là nội dung tự học có hướng dẫn, Sinh viên tự nghiên cứu thêm phần này

Kết luận





HCM-UTE

BÀI TẬP 1

Yêu cầu thực hành: Sử dụng biểu đồ tĩnh và dạng tương tác.

- Vẽ Box Plot:** Trực quan hóa biến thu nhập/hóa đơn; xác định mốc Q1, Q2, Q3, IQR và danh sách điểm dị biệt ($> 1.5 \times IQR$).
- Vẽ Histogram:** Phân tích điểm số/chi tiêu; thử nghiệm điều chỉnh số (bins = 10, 20, 50) và giải thích lý do không dùng Bar Chart cho dữ liệu liên tục.
- Vẽ Split Violin Plot:** Thực hiện so sánh sự khác biệt phân phối chi tiêu giữa Nam và Nữ.

Dataset sử dụng:

Yêu cầu 1,2: <https://kaggle.com/datasets/ruchi798/data-science-job-salaries>

Yêu cầu 3: Tips Dataset

Các yêu cầu phải được lập trình và chạy thực thi với kết quả trực quan ngay trong cùng 1 file notebook duy nhất

Cú pháp nộp: [hovatenSV_MSSV_chuong3_BT1.ipynb]



HCM-UTE

BÀI TẬP 2

Yêu cầu 1: Tạo sinh / Thu thập dữ liệu:

Sinh viên chọn 1 trong 2 hình thức:

Option 1: Viết vòng lặp Python sinh **500 giao dịch** có cấu trúc phân cấp 2 tầng (Category → SubCategory) kèm 4 chỉ số định lượng:

Sales (100–1000), Profit (phụ thuộc % theo Sales), Discount (0–30%), Quantity (1–10).

Option 2: Tải 1 dataset bán hàng/bất động sản bất kỳ từ Kaggle/Open Data và viết code lọc ra đúng 2 biến phân cấp và 4 biến định lượng.

Yêu cầu 2: Trực quan hóa dữ liệu dạng tĩnh và tương tác:

Task 2.1 (Correlation Heatmap): Tính ma trận tương quan Pearson giữa 4 biến định lượng. Vẽ Heatmap tương tác bằng plotly.express → hiển thị chỉ số r lên ô.

Task 2.2 (Hierarchical Treemap): Vẽ Treemap phân cấp (Category → SubCategory).

Mã hóa diện tích ô = tổng Sales và màu sắc = tổng Profit.

Lưu ý: Các yêu cầu phải được lập trình và chạy thực thi với kết quả trực quan ngay trong cùng 1 file notebook duy nhất

Cú pháp nộp: [BTNhom_hovatenSV_MSSV_chuong3_BT2.ipynb]



BÀI TẬP 3

Yêu cầu 1: Tạo sinh dữ liệu

Anh (chị) hãy viết script Python sinh bảng dữ liệu chuỗi thời gian **12 tháng** gồm các cột sau:

- Cột thời gian: Tháng (Tháng 1 đến Tháng 12).
- Cột số tuyệt đối: DoanhSo (tạo ngẫu nhiên từ 400 đến 900 triệu).
- Cột số tương đối: TyLeTangTruong_% (tính % thay đổi doanh số so với tháng trước MoM).
- Cột chỉ tiêu: KPI_Muctieu (gán giá trị cố định = 650 triệu cho tất cả các tháng).
- Cột bổ trợ: TopProduct (gán ngẫu nhiên tên sản phẩm bán chạy nhất tháng).

Yêu cầu 2: Tác vụ Trực quan hóa tương tác

Task 3.1: Vẽ biểu đồ kết hợp Trục Y1 bên trái (Cột: DoanhSo) và Trục Y2 bên phải (Đường: TyLeTangTruong_%).

Task 3.2: Chèn đường Reference Line tại mốc KPI_Muctieu và tùy chỉnh Hover Tooltip dạng HTML hiển thị trọn vẹn: *Tháng, Doanh số, % Tăng trưởng và TopProduct*.

Lưu ý: Các yêu cầu phải được lập trình và chạy thực thi với kết quả trực quan ngay trong cùng 1 file notebook duy nhất

Cú pháp nộp: [BTNhom_hovatenSV_MSSV_chuong3_BT3.ipynb]



HCM-UTE

BÀI TẬP 4

Bước 1: Thu thập / Định nghĩa dữ liệu không gian

Anh (chị) tự định nghĩa một DataFrame gồm **5 đến 10 Tỉnh/Thành phố** tại Việt Nam:

- Tra cứu và điền tọa độ địa lý: KinhDo (Longitude) và ViDo (Latitude).
- Tự sinh chỉ số kinh doanh: **DoanhThu** (tỷ đồng), **SoChiNhanh** (số lượng), và **VungMien** (Bắc/Trung/Nam).

Bước 2: Tác vụ Trực quan hóa & Tích hợp

Task 4.1 (Geospatial Bubble Map): Sử dụng Plotly (scatter_mapbox hoặc scatter_geo) vẽ các điểm thành phố lên bản đồ. Mã hóa kích thước bong bóng theo **DoanhThu** và màu sắc theo **VungMien**.

Task 4.2 (Mini Interactive Dashboard): Xây dựng một trang Dashboard đơn giản (bằng tính năng Subplots Plotly và ipywidgets) chứa 2 biểu đồ:

Bản đồ không gian (Task 4.1) + Biểu đồ Cột doanh thu theo Vùng miền, cho phép người dùng chọn vùng miền để lọc dữ liệu hiển thị.

Lưu ý: *Các yêu cầu phải được lập trình và chạy thực thi với kết quả trực quan ngay trong cùng 1 file notebook duy nhất*

Cú pháp nộp: *[BTNhom_hovatenSV_MSSV_chuong3_BT4.ipynb]*



THANK YOU !